



**SCHOOL OF ENGINEERING
ENGG341 Individual Project**

**Application of ML for Modelling and
Simulation of Nuclear Reactors
Final Report**

Joseph Dehghani
Civil Engineering
April 2026

Supervisor: Dr Dzianis Litskevich
Project No. 183

**School of Engineering
University of Liverpool
Brownlow Hill, Liverpool, L69 3GH**

Abstract

The rapid deployment of Small Modular Reactors (SMRs) is currently hindered by the immense computational expense of the Monte Carlo (MC) simulations required for regulatory validation. To overcome this bottleneck, interpretable, tree-based machine learning surrogate models were developed to predict the effective multiplication factor (k_{eff}) of a commercial SMR. A high-fidelity ground-truth dataset of 1 000 static reactor states was generated using the OpenMC Python API and the MIT NuScale-like ExaSMR benchmark. Subsequently, Random Forest Regressor (RFR) and eXtreme Gradient Boosting (XGBoost) models were trained and optimised using on the pre-processed dataset. The modern XGBoost architecture demonstrated a mean uncertainty of 336.8 pcm, while reducing prediction times to microseconds; providing a computational speed-up factor of 2 500 000 over the MC baseline. Furthermore, Gini impurity-based feature importance confirmed that the models successfully learned the underlying physical hierarchy of reactor parameters. However, it is concluded that while tree-based surrogates can effectively resolve computational bottlenecks for reactor simulations and provide superior transparency compared to opaque neural networks, their lack of explicit explainability restricts them from satisfying strict regulatory principles for safety-critical SMR applications. Therefore, these models are currently best suited for accelerating the timescales of preliminary designs and non-safety-critical studies.

Table of Contents

Abstract	ii
Table of Contents.....	iii
List of Figures	v
List of Tables.....	v
Nomenclature	vi
1 Introduction	1
1.1 Background	1
1.2 Aims and Objectives	2
2 Literature Review	3
2.1 The Regulatory Hurdle and Computational Bottleneck	3
2.2 Machine Learning in Nuclear Engineering.....	4
2.3 The “Black Box” Dilemma and Data Scarcity	6
2.4 Tree-Based Architectures and the Research Gap	7
3 Methodology	9
3.1 Experimental Design & Apparatus.....	9
3.2 High-Fidelity Stochastic Base-Data Generation.....	10
3.3 Data Preprocessing.....	18
3.4 Surrogate Model Architecture and Training.....	19
3.5 Scope and Assumptions.....	22
4 Results	24
4.1 Baseline Model Performance.....	24
4.2 Surrogate Model Performance	25
4.3 Surrogate Model Interpretability	28
4.4 Computational Speed-Up	29
5 Discussion	30
5.1 Interpretation of Results.....	30
5.2 Comparison Against Existing Studies	31
5.3 Regulatory and Safety Implications.....	32
6 Reflective Statement	33
6.1 Societal and Environmental Impact	33
6.2 Ethics and Safety	33
7 Conclusions.....	34

7.1 Conclusion	34
7.2 Future Work.....	35
Bibliography.....	37
Appendices	41
Appendix A: Project Package Layout and User Guides	41
Appendix B: Updated Risk Register.....	42
Appendix C: Initial vs Actual Project Gantt Charts	43
Appendix D: Generative AI Assisted Plots	43

List of Figures

Figure 1: Project Approach Flowchart.....	9
Figure 2: Architecture Diagram of the Automated Dataset Generation Pipeline, including all file interactions.	15
Figure 3: Spearman Correlation Heatmap of Physical Reactor Parameters.	18
Figure 4: Distribution of the Target Variable and its Corresponding MC Standard Deviation for the Baseline Dataset.....	24
Figure 5: Parity Plots Evaluating the Predictive Performance of the Surrogate Models against the MC Ground-Truth.	26
Figure 6: Violin Plot Showing the Probability Density of Error in the ML Model Predictions.	27
Figure 7: Gini Feature Importance Ranking of Reactor Input Parameters for RFR and XGBoost Surrogate Models.....	28
Figure 8: Surrogate Model Predictions vs. Monte Carlo Ground-Truth for Two Dominant Features.	29
Figure 9: Initial and Actual Project Gantt Charts from the Project Proposal Stage (17/10/25) to Submission (24/04/26).	43

List of Tables

Table 1: Local Hardware Specifications.....	9
Table 2: Input Parameter Boundaries and Influence on $keff$	12
Table 3: Simulation Parameters and their Effects on Fidelity and Computation.	16
Table 4: Tested Hyperparameter Values for the 5-fold Cross-validation GridSearch.....	21
Table 5: Best GridSearch Hyperparameters and RMSE for the ML Surrogate Models.....	25
Table 6: Summary of Predictive Uncertainty and Computational Speed-Up for the Evaluated Reactor Models.....	29
Table 7: Final Project Risk Register.	42

Nomenclature

Mathematical Symbols:

f	Thermal utilisation factor
k_{eff}	Effective neutron multiplication factor
p	Resonance escape probability
β	Delayed neutron fraction

Acronyms:

ALARP	As Low As Reasonably Practicable
ARO	All Rods Out
ARI	All Rods In
BOL	Beginning of Life
BWR	Boiling Water Reactor
CR	Control Rod
DT	Decision Tree
GDA	General Design Assessment
LHS	Latin Hypercube Sampling
MC	Monte Carlo
ML	Machine Learning
NN	Neural Network
NPM	NuScale Power Module
ONR	Office for Nuclear Regulation
pcm	Percent mille (parts per hundred thousand)
ppm	Parts per million
PWR	Pressurised Water Reactor
RFR	Random Forest Regressor
RMSE	Root Mean Squared Error
SAPs	Safety Assessment Principles
SMR	Small Modular Reactor
XAI	eXplainable Artificial Intelligence
XGBoost	eXtreme Gradient Boosting

1 Introduction

1.1 Background

The UK's commitment to achieving Net Zero by 2050 (*The Climate Change Act 2008 (2050 Target Amendment)*, 2019) requires a massive expansion of reliable, low-carbon power generation. To meet this infrastructural demand, the civil nuclear sector is rapidly shifting towards the deployment of Small Modular Reactors (SMRs); with the UK aiming to deploy up to 24GW of SMR capacity by 2050 (Department for Energy Security and Net Zero, 2024). SMRs are nuclear reactors designed for rapid production and deployment. They are an emerging technology requiring real-world deployment to assess value for money and delivery certainty (Department for Energy Security and Net Zero, 2024). By utilising the strengths of factory modularisation and rapid site assembly, SMRs offer a highly scalable and economically viable solution compared to traditional nuclear reactors.

In a traditional Pressurised Water Reactor (PWR), Uranium-235 nuclei in the fuel rods absorb slow moving neutrons, splitting via fission to release thermal energy and fast-moving neutrons. These fast-moving neutrons are slowed down by a light water moderator to sustain a stable chain reaction within the fuel rods. Engineers actively control this chain reaction using neutron absorbing interventions such as control rods and soluble boron dissolved in the coolant/moderator (Mercier, 2025). Furthermore, a reactor core naturally stabilises itself against large power spikes through the Doppler effect, where hotter fuel absorbs neutrons before they can be moderated to cause fission. Temperature spikes within a PWR also affect the moderator temperature coefficient, where hotter, less dense light water fails to moderate neutrons, naturally dropping the fission within the fuel to a stable state (Mercier, 2025). Modern reactor designs harness these physical phenomena to autonomously sustain a nuclear chain reaction, producing thermal energy that can be utilised for low-carbon energy generation.

A prime example of SMR technology is the NuScale Power Module (NPM), a small, light-water-cooled pressurized water reactor (PWR) designed for passive safety. Because NPMs rely on being submerged in containment pools for natural water circulation (Reyes and Ingersoll, 1996) rather than complex mechanical pumps, the fabric of their design inherently serves as their primary safety mechanism, allowing for unlimited coping time after a design basis accident without any intervention (Subki, 2020). However, to legally validate these novel systems without physical testing, engineers rely heavily on computationally expensive Monte Carlo (MC) computer

simulations, creating a significant bottleneck in satisfying pre-deployment regulatory standards (Subki, 2020).

Machine Learning (ML) offers an innovative solution to this limitation. By training an ML model on extensive pre-computed Monte Carlo datasets, the model learns the underlying correlations between physical reactor parameters, such as the effective multiplication factor (k_{eff}). A trained surrogate model bypasses the complex physics-based simulations of Monte Carlo by using pattern recognition rather than simulating thousands of individual particles. This offers a significant computational upgrade (Alizadeh, Allen and Mistree, 2020). Consequently, with a robust ground-truth dataset, ML models have the potential to conduct real-time reactor simulations, accelerating the deployment of SMRs. However, before these algorithms can be deployed, their decision-making process must be rigorously assessed to ensure that they satisfy strict safety assessments mandated by the Office for Nuclear Regulation (ONR).

1.2 Aims and Objectives

1.2.1 Primary Aim

The primary aim of this project is to develop an interpretable Machine Learning surrogate model capable of predicting the criticality (k_{eff}) of a Small Modular Reactor (SMR). By reducing the computational demand required for reactor simulations this project aims to evaluate if surrogate models can bridge the gap between computational speed and strict regulatory standards.

1.2.2 Objectives

- 1) To overcome the lack of open-source SMR data by independently generating a high-fidelity Monte Carlo (MC) dataset.
- 2) To develop a data-preprocessing pipeline to clean and format the raw MC data for Machine Learning (ML) applications.
- 3) To train and optimise interpretable ML surrogate models to predict the effective multiplication factor (k_{eff}) based on the pre-processed MC dataset.
- 4) To critically evaluate the ML surrogate models against the MC ground-truth, specifically analysing the trade-off between predictive accuracy and computational speed-up.
- 5) To evaluate the interpretability of the ML models, assessing their viability for compliance with strict ONR regulations.

2 Literature Review

2.1 The Regulatory Hurdle and Computational Bottleneck

Despite the inherent safety and economic advantages of SMRs outlined in Sec. 1.1, their deployment faces significant regulatory burdens. The DESNZ roadmap to 2050 (Department for Energy Security and Net Zero, 2024) recognises this regulatory friction withholding SMRs from service, explicitly targeting “reducing the timescales for the completion of a Generic Design Assessment (GDA) by up to 50%” (Department for Energy Security and Net Zero, 2024, pp. 7, 37). However, despite DESNZ’s target of streamlining the regulatory approval process, the Office for Nuclear Regulation (ONR) enforces the As Low As Reasonably Practicable (ALARP) principle as the nuclear sector’s alignment with the legal duty to reduce risks “so far as reasonably practicable” (Office for Nuclear Regulation, 2020, p. 9). ALARP applies to all levels of risk, “extending below the broadly acceptable level” (Office for Nuclear Regulation, 2020, p. 9) as set out by the ONR’s Safety Assessment Principles (SAPs).

The SAPs set out a “gross disproportion” test ensuring that dutyholders prove that they have identified all possible measures to control risk, and thoroughly justify why the cost, time and effort of alternative improvements would be “grossly disproportionate” to the actual safety benefit gained (Office for Nuclear Regulation, 2020, pp. 150–151). Because ALARP enforces “defence in depth against potentially significant faults” (Office for Nuclear Regulation, 2020, p. 38), novel approaches such as the NPM must satisfy an extensive GDA involving Design Basis Accident Analysis (DBAA) to ensure that a reactor design can withstand a worst-case scenario, alongside Probabilistic Safety Assessment (PSA) to quantify the plant’s overall risk during operation (International Atomic Energy Agency, 2007).

This combination of regulatory burdens placed on the adoption of SMRs necessitates rigorous pre-deployment validation. However, physical testing of severe fault conditions is highly impractical for nuclear reactors, making computational analysis the only option. Therefore, GDA safety cases mandate that all possible reactor states are pre-emptively tested using high-fidelity simulations (Office for Nuclear Regulation, 2025).

The primary objective of high-fidelity reactor safety simulations is to continuously monitor the effective neutron multiplication factor (k_{eff}) to maintain a stable chain-reaction and prevent prompt criticality (Mercier, 2025). However, calculating k_{eff} is highly complex because it relies on tracking the whole neutron lifecycle across multiple interacting probabilities governed by the six-factor formula (Mercier, 2025, p. 18).

To mathematically compute the complex neutron lifecycle within a reactor, software must solve the “Boltzmann neutron transport equation” (Mercier, 2025, p. 37). Historically, engineers have utilised deterministic methods involving simplified mathematical models to calculate k_{eff} . In addition to being mathematically complex, deterministic methods rely on significant mathematical approximations, which heavily restrict their accuracy (Barrientos and Alfonso, 2021), making them fundamentally inaccurate for novel SMR designs. In contrast, stochastic Monte Carlo (MC) methods are the undisputed benchmark for reactor physics simulations because they solve the neutron transport equation by “randomly sampling the physical interaction of neutrons in a material according to its reaction cross sections” (Barrientos and Alfonso, 2021, p. 2). OpenMC specifically stands out as the industry standard software for the stochastic method as it utilises “continuous-energy cross sections” and uses “exact solid geometry” (Romano *et al.*, 2015, p. 91) to model a reactor exactly as it exists in reality with no error.

Despite the unparalleled fidelity of MC simulations, they are fundamentally limited by the massive computational burden they impose. Their core limitation is that the statistical uncertainty of MC simulations converges slowly, scaling inversely to the square root of the number of simulated particles ($1/\sqrt{N}$) (Barrientos and Alfonso, 2021, p. 2). Therefore, achieving a reliable estimate requires simulating millions of independent particles, directly implying that “MC simulations are very time consuming” (Barrientos and Alfonso, 2021, p. 20). Barrientos and Alfonso (2021) provides a tangible example: Simulating only 50 seconds of real-world reactor time with a high enough accuracy results in “216 days of continuous wall-clock computing time” (Barrientos and Alfonso, 2021, p. 81). Although the unparalleled accuracy of MC methods makes this approach legally robust, their computational demand renders them practically unviable for the real-time operational safety monitoring required by SMRs.

2.2 Machine Learning in Nuclear Engineering

The proposed solution to the computational bottleneck in MC methods is the Machine Learning (ML) surrogate model, which is defined as an approximate analytical representation used to “replace expensive simulation models of engineering problems” (Alizadeh, Allen and Mistree, 2020, p. 2). ML surrogates bypass computationally intensive physics-based calculations by learning correlations from pre-computed databases rather than incrementally solving complex equations. They are an alternative model that yields estimated output values (targets) directly from inputs (features) using Machine Learning (ML) architectures. Recent efforts have been made

to develop fast-running surrogate models for nuclear applications, although progress is still possible in this field to simplify the calculation of reactor parameters (Zhang *et al.*, 2025).

Recent literature has proven that the theoretical surrogate model concept has already achieved promising computational gains (Huang *et al.*, 2023), particularly when applied to large-scale commercial reactors. Oktavian (2023) studied how ML could be applied to the simulation of Boiling Water Reactors (BWRs) as a strategy to overcome the computational bottleneck. This study used a visual-based neural network (Convolutional Neural Network) to learn the correlations between low-fidelity and high-fidelity BWR cross sections to reduce the uncertainty for low-fidelity simulations in routine operations. The study found that 528 CPU-hours of high-fidelity Monte Carlo generation could be reduced to 20.1 s of low-fidelity modelling paired with the CNN model. In another study, Gong *et al.* (2022) implemented Decision Tree (DT) and K-Nearest Neighbour (KNN) models, both supervised learning models based on numerical input parameters (such as moderator temperature), to act as digital twins of the Hualong One (HPR1000) pressurised water reactor (PWR). The study found that 35 s of deterministic modelling time was reduced to <0.03 s by both ML models. These results indicate that numerical supervised learning models can be extremely effective in reducing reactor modelling times by several orders of magnitude.

In addition to commercial applications, ML-based surrogate models have proven to be highly effective for smaller experimental reactor architectures. Researchers at the Chinese Academy of Sciences used back-propagation neural networks (BPNN) to predict for the Molten-Salt Reactor Experiment (MSRE), a small experimental reactor using liquid fuel (Robertson, 1965). The study found that a high-fidelity OpenMC simulation took 77 min, whereas the BPNN-based surrogate model could predict in only 0.002 s (Bei *et al.*, 2023). This represents a speed up of 2.31 million times. Additionally, Sanchez and dos Santos (2021) implemented a Support Vector Regression (SVR) model to simulate an experimental boron-free SMR. The study found that the supervised learning-based SVR model could significantly reduce complexity while maintaining a Root Mean Squared Error (RMSE) of around 0.1% (equivalent to 100 pcm). Across the literature, an uncertainty of less than 100 pcm is targeted as the standard benchmark for highly accurate predictive models. This indicates that ML-based surrogate models are not only computationally efficient but can also produce an uncertainty in the results that is low enough for routine operations.

At both commercial and experimental/modular reactor scales, ML has proven to be an effective tool for combating the computational bottleneck caused by the sheer physical complexity of

reactor simulations. To be legally viable for real-world applications, these models must not only be fast and accurate, but mathematically transparent.

2.3 The “Black Box” Dilemma and Data Scarcity

While the industry is heavily invested in neural networks (like CNNs and BPNNs) for their speed, their deployment for real world applications is fundamentally incompatible with UK nuclear regulations. Under the ALARP framework, reactor operators bear a legal duty to “understand how an algorithm arrives at its conclusions, whether AI-guided or not” (Hall and Agarwal, 2024, p. 4). The reality is that neural network-based surrogate models act as opaque “black boxes” (Huang *et al.*, 2023, p. 10). Their decision-making process is highly complex, based on the non-linear interaction between millions of parameters that are almost impossible for humans to interpret. This lack of transparency directly violates the ONR’s Safety Assessment Principles, which mandate the need for physical interpretability (Office for Nuclear Regulation, 2020). Furthermore, because their internal logic is hidden behind millions of parameters, regulators currently have no mathematical method of quantifying the failure probability of NNs, making it difficult to trust their predictions (Office for Nuclear Regulation, CNSC and USNRC, 2024, p. 15).

On top of the “black box dilemma”, NN-based surrogate models face an additional logistical flaw: they mathematically require massive amounts of data to establish reliable patterns (Gong *et al.*, 2022). Owing to the high cost and long lifecycle of experiments, real-time reactor data is scarce, even non-existent for experimental reactor designs. Therefore, to create a ground-truth dataset, nuclear engineers have to use traditional high-fidelity simulations such as MC methods. Both Bei *et al.* (2023) and Oktavian (2023) agree that the high-fidelity neutron transport calculations required for training their NN-based models came with significant computational costs. Because high-fidelity data is so computationally and financially expensive, researchers are faced with severe data scarcity. In a recent study, Tao *et al.* (2025) highlighted the severity of this trap, estimating that 90% of all ML datasets used in material science studies had less than 1 000 samples, while also stating that 1 000 samples is generally considered insufficient for NN-based architectures (Tao *et al.*, 2025, p. 387).

When a data-hungry NN is forced onto a small nuclear dataset with many features, its reported accuracy can become a dangerous illusion. The surrogate model memorises noise in the limited dataset, disconnecting it from the fundamental physics provided by the high-fidelity simulations; this is known as overfitting (Tao *et al.*, 2025). Overfitting is particularly important to avoid in nuclear applications under the ALARP principle as it causes the model to overperform on test

data but fail for new data, spoiling the verifiability of the model. While ML-based models can successfully interpolate training data, Basu *et al.* (2022) notes that they extrapolate data poorly. This means an ML model cannot safely predict phenomena outside the range of its training data. Instead of failing safely, these reactor anomalies can cause NN-based models to collapse, outputting flawed results with “overconfidence” (Basu *et al.*, 2022). This blind overconfidence, paired with the opacity of the inner workings of a NN could hide potentially dangerous errors from operators, further compromising the in-depth verifiability mandated by the ONR.

Initiatives to bridge the gap between NNs and interpretability exist, like Explainable Artificial Intelligence (XAI), which holds NNs to the same standards of explanation as human decision makers (Hall and Agarwal, 2024). However, XAI is intrinsically dependent on extensive historical data and expertise. This renders it largely useless for the novel reactor designs common in SMRs that lack the benefit of extensive documentation (Hall and Agarwal, 2024). Consequently, the nuclear industry requires a surrogate modelling architecture that retains the computational speed-up of machine learning but provides verifiable results and resistance to overfitting on sparse datasets.

2.4 Tree-Based Architectures and the Research Gap

To satisfy strict regulations and overcome data scarcity, the industry should transition from opaque Neural Networks to tree-based machine learning models. Models based on decision trees like the Random Forest Regressor (RFR) are inherently transparent and easy to interpret (Tao *et al.*, 2025, p. 390). Beyond this, these models directly measure the physical impact of each feature, known as feature importance (Breiman, 2001). This means that RFRs can identify exactly which features of a nuclear dataset are most effective at predicting deviations in k_{eff} . While other supervised learning models like the kernel-based Support Vector Regressors (SVRs) exist, they lack this highly interpretable feature importance. Unlike NNs and SVRs, this transparent feature importance provides the mathematical transparency required to evaluate whether a tree-based surrogate model can satisfy the ONR’s ALARP principles.

An additional benefit of a RFR is its immunity to the overfitting problem that plagues NNs with small datasets. It does this through its Bootstrap Aggregating (Bagging) architecture which generates multiple data subsets and trains independent models (trees) based on them. By aggregating the results of these subsets, the RFR reduces the bias and variance of the overall model (Tao *et al.*, 2025, p. 389). Since this algorithm relies on the Strong Law of Large Numbers, the RFR model converges to a stable limit as the number of trees increases; meaning they do not

overfit and inherently possess the ability to safely generalise extrapolated inputs like reactor anomalies (Breiman, 2001, pp. 7, 29). The RFR's bagging architecture acts as a shield protecting the surrogate model from the data scarcity problem plaguing NN-based models.

A modern development of tree-based ML models is the Extreme Gradient Boosting (XGBoost) algorithm created by Chen and Guestrin (2016). XGBoost is a tree-based ML algorithm that learns from the failures of previous trees to maximise the speed and accuracy of a tree-based surrogate model (Devan and Khare, 2020). Thanks to its fast execution and improved accuracy, XGBoost has been found to be a state-of-the-art tool in exceeding traditional models for nuclear applications (Huang *et al.*, 2023). However, literature suggests that boosting algorithms may be more susceptible to noise in sparse datasets than bagging methods (Alizadeh, Allen and Mistree, 2020). This raises the question of how the XGBoost algorithm will perform compared to the simpler RFR algorithm for predicting k_{eff} with the limited MC datasets typical of nuclear applications (Barrientos and Alfonso, 2021).

While OpenMC is widely regarded as the gold standard for high-fidelity data generation, the literature reveals it is currently being misapplied to opaque surrogate models (like NNs) for non-commercial reactors. Conversely, when interpretable, tree-based models (such as DTs and KNNs) are utilised, researchers restrict them to lower-fidelity deterministic datasets. Therefore, there is a critical lack of research applying transparent tree-based ML models to predict operational safety parameters (like k_{eff}) in commercial SMRs using high-fidelity OpenMC data.

This project aims to bridge this gap by comparing the traditional RFR baseline against the modern XGBoost algorithm to determine whether boosting architectures offer improvements on sparse nuclear datasets without sacrificing transparency and robustness for compliance with ONR regulations. This evaluation will establish whether modern tree-based architectures represent a viable advancement over traditional ML architectures for operational SMR applications.

3 Methodology

This chapter details the three-stage computational pipeline developed to generate, preprocess and evaluate the tree-based ML surrogate models. The three-stage pipeline followed by an evaluation of results is visualised in Fig. 1. To maintain project management, the Project Risk Register, provided in App. B, was consistently monitored and updated throughout the process of this methodology. To ensure full reproducibility, the full project package and user guides are provided in App. A.

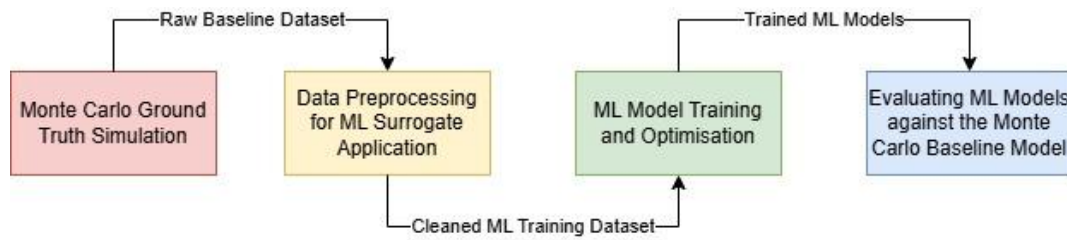


Figure 1: Project Approach Flowchart.

3.1 Experimental Design & Apparatus

To resolve the computational and regulatory bottlenecks established in Ch. 2, the methodology was structured as a three-stage pipeline: 1) Stochastic base-data generation, 2) Data Preprocessing, 3) Surrogate model training and validation.

3.1.1 Physical Apparatus

The computation for both the stochastic (OpenMC) model and the surrogate (RFR) model was executed locally on a PC with the specifications detailed in Tbl. 1.

Table 1: Local Hardware Specifications.

Type	Specification
CPU	Intel i7-1260P (12 cores, 16 threads, 2.1 GHz)
RAM	8 GB
Operating System	Windows 11/ Windows Subsystem for Linux 2 (WSL2) via Ubuntu

Both the traditional MC simulation and the ML surrogate models were run on the exact same machine. Since the hardware is kept constant, any large difference in computation time is undeniably due to the efficiency of the ML algorithm, establishing a controlled baseline for computational cost. OpenMC natively utilises OpenMP multithreading (Romano *et al.*, 2015),

automatically using all 16 available CPU threads. Despite maximising hardware utilisation, the Monte Carlo method still proved too computationally expensive for real-time application (described further in Sec. 3.2.6).

3.1.2 Software Stack

All software versions were strictly managed to guarantee the reproducibility of this methodology. A Conda Environment [Version 26.1.0] running the latest stable version of Python [Version 3.14.3] was used to manage computation across all three pipeline stages. Because OpenMC relies on native POSIX systems, Windows Subsystem for Linux 2 (WSL2) via Ubuntu was required to execute the base-data generation phase (Sec. 3.2).

The baseline reactor model was constructed using the MIT ExaSMR benchmark (Romano) and the OpenMC Python API [Version 0.15.3]. The MC simulations were executed using OpenMC (Romano *et al.*, 2015) paired with the highly verified ENDF/B-VII.1 nuclear cross-section library (Chadwick *et al.*, 2011). For the subsequent data preprocessing phase (Sec. 3.3), the data manipulation was handled using Jupyter [Version 7.5.5]. Finally, the surrogate models were constructed using Scikit-learn [Version 1.8] and the XGBoost library [Version 3.2.0] (Chen *et al.*, 2019). All visual plots were generated using Matplotlib [Version 3.10] and Seaborn [Version 0.13.2].

3.2 High-Fidelity Stochastic Base-Data Generation

The primary objective of this phase was to generate a robust ground-truth dataset that represents the complex operational parameters of a commercial SMR. This section details the baseline geometry, the selected input features and the automated OpenMC pipeline utilised to extract the target variable (k_{eff}).

3.2.1 The Baseline Model

The base OpenMC model used for this methodology was the MIT ExaSMR benchmark (Romano), an open-source 3D model mimicking the design of the NuScale power module (NuScale Power LLC, 2016). This package consists of a group of Python scripts (importantly `build-core-fresh.py` and the `/smr` folder) that utilise OpenMC's Python API to construct the geometry of a baseline core which is exported as a group of `.xml` files. Since the exact NuScale geometric and

material properties are proprietary (strictly controlled by the NRC), using an exact replica of a commercial SMR is impossible. However, the MIT model provides a pre-verified replica intended on capturing most of the physical complexities involved in modelling a full reactor core. This isolates the subsequent ML surrogate model from foundational geometric errors within the OpenMC model.

To compensate for the lack of proprietary data, the MIT ExaSMR model relies on geometric and material approximations. For example, specific NuScale fuel assembly details are approximated, and control rod insertion step heights are derived from a separate MIT reactor design: the Westinghouse BEAVRS benchmark (Horelik *et al.*, 2013). Additionally, the model assumes a Beginning of Life (BOL) fuel composition. Despite these limitations, the computational burden and physical complexity of the MIT model remain identical to reality, making it a suitable model for this application.

The ExaSMR benchmark acts as a static baseline model. To utilise it for generating a dataset, three core changes had to be made:

- The original model defined the geometry and material properties of control rods but left all guide tubes empty in its assemblies. The scripts `core.py` and `surfaces.py` in the `/smr` folder were altered to physically load control rods into the OpenMC assembly (detailed further in Sec. 3.2.3).
- The OpenMC Python API undergoes frequent updates affecting its notation. Since the open-source model was last committed in 2022, depreciated API code (such as `rectangular_prism` and `source.Source`) required updating to remain compatible with modern versions (OpenMC Development Team, 2024).
- To create physical alterations within the reactor model (necessary for having a diverse dataset for the surrogate model), a pair of code injection blocks were implemented into the base `build-core-fresh.py` script. These injections allowed the baseline model to be overwritten with the chosen input parameters (detailed in Sec. 3.2.2).

3.2.2 Input Parameters and Boundaries

To capture a varied dataset, specific input parameters (summarised in Tbl. 2) were selected to manipulate k_{eff} . To ensure the model remained physically viable, the input boundaries were specifically chosen based on the nominal operating conditions of the NuScale power module (NPM).

Table 2: Input Parameter Boundaries and Influence on k_{eff} .

Input Feature	Boundaries		Effect on k_{eff} via the 6 Factor Formula
	Lower	Upper	
Fuel temperature	500 K	1200 K	Increased fuel temperature reduces resonance escape probability (p).
Moderator temperature	500 K	600 K	Increasing moderator temperature decreases density.
Moderator Density *	0.65 g/cm ³	0.76 g/cm ³	Increased moderator density increases p and simultaneously decreases thermal utilisation (f).
Cladding Temperature	500 K	700 K	Negligible impact on k_{eff} .
Boron Concentration	0.01 mg/L (ppm)	1235 mg/L (ppm)	Increasing boron concentration lowers the thermal utilisation factor (f).
Control rod step	0 (ARI)	248 (ARO)	Thermal shadowing from control rods reduces k_{eff} .

* Not independently sampled; calculated dynamically based on Moderator Temperature at 12.76 MPa.

Increasing Fuel Temperature directly drives the Doppler effect, where U-238 resonances widen, capturing more neutrons and therefore reducing k_{eff} (Mercier, 2025, Ch. 15). The fuel temperature range of 500-1200K captures the temperature of the core from hot shutdown (488 K) to beyond the peak fuel pellet temperature of 1155 K (NuScale Power LLC, 2016, Tbl. 4.4-8).

The NPM utilises a combined coolant and moderator system using borated water. Increasing Moderator Temperature causes the fluid to expand, reducing the concentration of moderating atoms and trapping fast moving neutrons in the unreactive “epithermal range” (Mercier, 2025, Ch. 4). The 500 to 600 K boundary reflects core temperatures from hot shutdown up to a conservative limit just below the boiling point of water at NuScale’s nominal system pressure of 12.76 MPa (NuScale Power LLC, 2016, Tbl. 4.1-1). To ensure thermodynamic accuracy when changing the moderator temperature, Moderator Density was calculated dynamically before each simulation using the IAPWS library, deriving the exact water density based on the sampled temperature at 12.76 MPa (which was verified to be the system pressure of the MIT model via `__init__.py`) (Kretzschmar and Wagner, 2019).

The NPM’s cladding is composed of an M5 alloy and a Zirconium alloy (Zry-4). The 500 to 700 K cladding boundary covers the operational temperature range while remaining safely below the phase transformation temperature of both M5 and Zry-4, which is around 1 000 K (Kecek A et al., 2016, Tbl. 1).

Boron is the soluble material dissolved in the moderator to absorb thermal neutrons, serving as the primary long-term mechanism to control reactivity. The 1 235 ppm upper limit matches NuScale’s initial Boron Concentration at BOL (NuScale Power LLC, 2016, Sec. 4.3.2.1). As the fuel

material depletes, the Boron Concentration typically reaches 20 ppm or less (NuScale Power LLC, 2016, Sec. 4.3.1.1). Therefore, the lower boundary is set at 0.01 ppm to prevent a divide-by-zero error in OpenMC’s mathematical process.

Alongside boron concentration, control rods (CR) act as the primary method of controlling core reactivity. CRs introduce negative reactivity into the reactor core by absorbing neutrons, controlling k_{eff} (NuScale Power LLC, 2016, Sec. 4.1.1). The NPM organises its control rods into two groups: the regulating bank and the shutdown bank. The shutdown banks remain in an All Rods Out (ARO) state while the regulating bank is moved in steps to replicate real-world NuScale operation (NuScale Power LLC, 2016, Sec. 4.3.2.1). The MIT model assigns Bank D control rods as the regulating bank and the other banks A, B and C as shutdown banks. Additionally, the MIT model’s control rod insertion is divided into 248 steps at approximately 1.58 cm step width, derived from MIT’s Westinghouse BEAVRS benchmark to estimate the proprietary CR insertion stages of the NPM (explained further in Sec. 3.2.1).

3.2.3 Updating the Core Builder Script (Code Injections)

An audit of the `/smr` folder revealed that despite the MIT script including all relevant materials and geometries for the control rods, they were absent from the assemblies in the OpenMC model. The regulating CR bank D (described in Sec. 3.2.2) had to be added manually to the model assembly in `assemblies.py`. To dynamically manipulate these rods, the first code injection block was implemented into the core builder script (`build-core-fresh.py`) before the OpenMC geometry is constructed. Reading the sampled step from the input file, the insertion of the control rods was changed by altering the plane of the bottom of the rods (Z-plane) to change how far they reach into the reactor core:

$$Z_{bottom} = Z_{fully\ inserted} + (Control\ rod\ step \times Step\ size) \quad (1)$$

$$Z_{top} = Z_{bottom} + Control\ rod\ length \quad (2)$$

As the control rod step decreases, the Z-Plane boundaries of the control rods translate into the core, acting as the insertion of the bank D control rods. The use of a 1D model for CR insertion is verified in the literature (Mercier, 2025, p. 56). Furthermore, the CR guide tube dashpots were flattened to the bottom of the fuel rod to ensure that surface crossover errors do not occur when fully inserted rods touch the boundary at the end of the guide tubes (OpenMC Development Team, 2024).

Following the geometry construction, the second code injection block was implemented to update the material properties within the model. By auditing the `materials.py`, `surfaces.py` and `core.py` scripts in the `/smr` folder, the specific string names used for each target material were extracted. The injection script iterates through each material (such as Zircaloy-4), recognising each relevant material with substring matching to ensure it captures materials in batches (such as different fuel pin enrichments). Once identified, the specific material's property is updated with the sampled input parameter (discussed in Sec. 3.2.2) using OpenMC API object modification (e.g. `mat.temperature`). This injection was strategically placed after the geometry is mapped, but before the model is saved, ensuring all altered properties are successfully exported to the `.xml` model geometry files.

For soluble boron, the MIT code requires inputs as an atomic fraction rather than the sampled parts per million (ppm). The `materials.py` script provides the mathematical process of calculating this using OpenMC's `atomic_weight` API. Crucially, before the boron concentration is updated in OpenMC, the `mat._nuclides.clear()` command clears the existing baseline nuclides to ensure they don't layer on top of the newly calculated nuclides in the final model (OpenMC Development Team, 2024).

3.2.4 Sampling Strategy

To gather a diverse combination of input variables, the SciPy Latin Hypercube sampler (LHS) was chosen over a simple random number generator, such as `np.random`. The LHS divides each parameter's range into non-overlapping intervals to prevent clustering (Virtanen *et al.*, 2020). This ultimately lowers the number of simulations required to train a surrogate model, without sacrificing the quality of data (Alizadeh, Allen and Mistree, 2020).

The normalised LHS matrix (between 0 and 1) was scaled to the upper and lower parameter boundaries (defined in Sec. 3.2.2), creating a single matrix storing every input sample. These were then unpacked into the individual variable arrays to be applied to the Monte Carlo model. Moderator density was not included in the LHS because it was dynamically calculated during each iteration based on the sampled moderator temperature. While the temperatures and boron concentration are continuous numbers, the control rod steps are discrete. Consequently, the continuous LHS outputs for the CR steps were rounded to the nearest integer utilising NumPy's `np.round` and `.astype(int)` functions (Harris *et al.*, 2020). This ensures that the control rods were not positioned mid-step.

3.2.5 Automated Pipeline Architecture

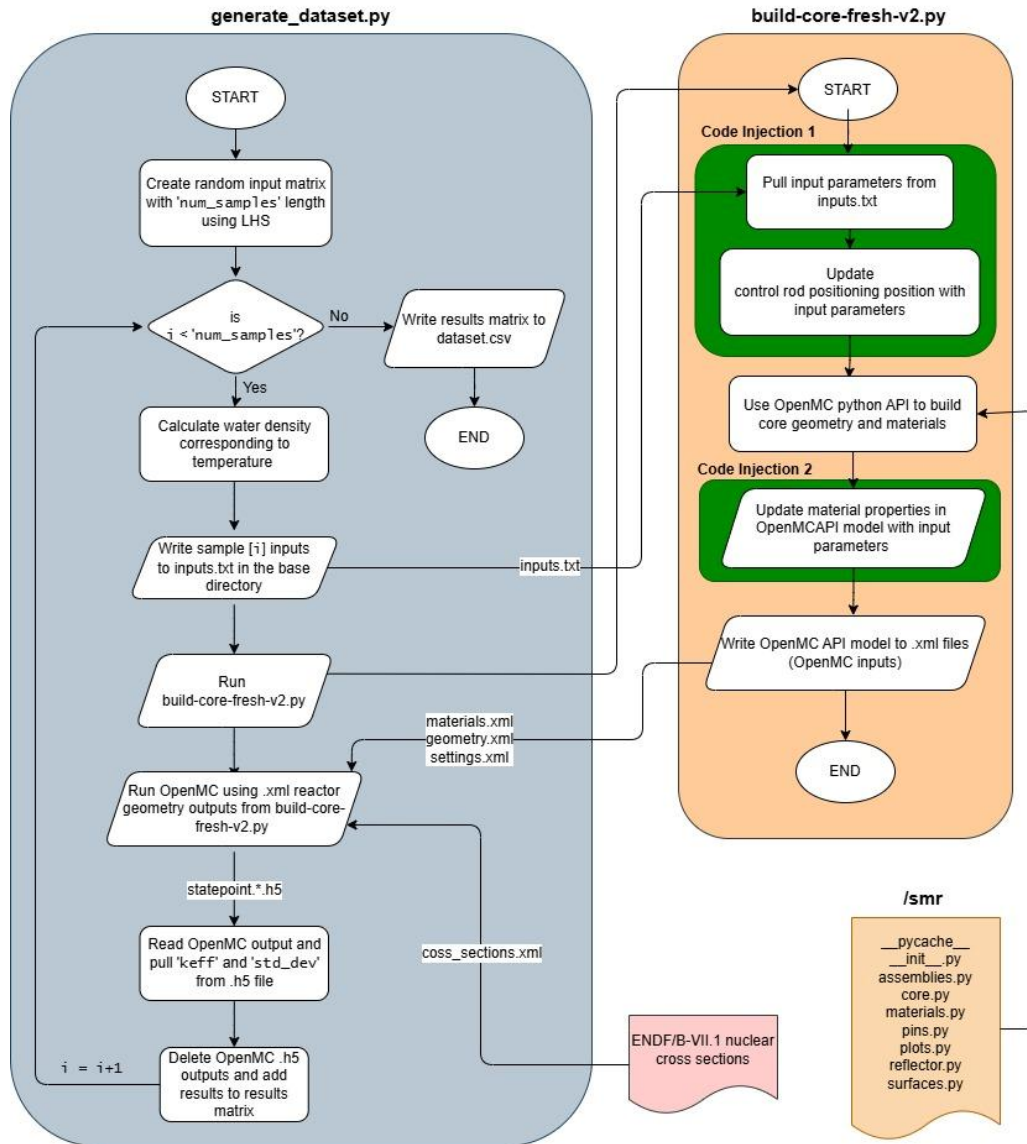


Figure 2: Architecture Diagram of the Automated Dataset Generation Pipeline, including all file interactions.

As illustrated in Fig. 2, the data generation pipeline was governed by the overarching script, `generate_dataset.py`. This script initialised the LHS input matrix and executed an automated loop to simulate each sample sequentially.

For each iteration, the loop first dynamically calculated the moderator density using the IAPWS library (Sec. 3.2.2). The script then wrote the current sample's parameters to an `inputs.txt` file, allowing them to be read by the `build-core-fresh-v2.py` script. Upon execution of the updated core-builder script, the first injection block altered control rod insertion, the MIT script constructed the model geometry, and the second injection block updated the material properties before outputting the final `.xml` geometry files to a new folder, `/core-fresh`.

With the geometry output from `build-core-fresh-v2.py` and the ENDF/B-VII.1 nuclear cross-sections, the overarching script ran the OpenMC simulation and extracted the resulting target (k_{eff}) and its standard deviation. Finally, the loop deleted the large OpenMC output files, preventing the process from accumulating redundant data, which would otherwise quickly fill storage.

Once done, the overarching script reformatted the results matrix, using the pandas library (McKinney, 2010) and saved them to a `.csv` dataset. To ensure a robust design, the loop was wrapped in a `try/except/finally` block, guaranteeing that the results were always saved if interrupted. Additionally, `if/continue` blocks were used throughout the script to skip possible errors (such as OpenMC failing to converge on k_{eff}), ensuring the automatic loop did not fail mid run. Another measure to ensure the loop was not interrupted was by running `generate_dataset.py` with the `nohup` (no hang up) Linux command, which continues to run the script even if the computer sleeps or the terminal closes.

3.2.6 Simulation Parameter Tuning

A dataset of 1 000 samples was targeted to train the ML models. While Tao *et al.* (2025) noted that 90% of material science ML studies use datasets of <1 000 samples, this represents the upper boundary of what is considered small. Consequently, this limitation necessitated careful hyperparameter tuning to avoid overfitting, particularly for the XGBoost model.

Table 3: Simulation Parameters and their Effects on Fidelity and Computation.

Parameter	Chosen Value	Primary Impact
Number of Samples	1,000	The dataset length required for training a reliable ML surrogate.
Simulated Particles	10,000 / batch	Primary driver for k_{eff} standard deviation.
Active Batches	200	Same effect on k_{eff} as particles but requires significantly more computation.
Inactive Batches	50	Ensures that the simulation converges.
Annular Rings	1	Eliminates redundant models for burnup calculations.
Axial Subdivisions	5	Retains sufficient control rod resolution while reducing computation.

Since the standard deviation of MC simulations regresses proportionally to $1/\sqrt{N}$, the number of active batches and simulated particles were the primary drivers for the fidelity of the baseline model. While increasing either parameter has the same effect on standard deviation, increasing batches is far more computationally expensive than simulating more particles (Romano *et al.*,

2015). Therefore, a minimum of 10 000 particles per batch was selected following similar studies (Barrientos and Alfonso, 2021). To produce an acceptable mean standard deviation, this particle count was paired with 200 active batches (Gill, Griesheimer and Aumiller, 2017). Furthermore, 50 inactive batches were required to ensure the model fully converged before it started collecting data (Barrientos and Alfonso, 2021).

The original MIT ExaSMR benchmark modelled the fuel pins with 10 annular rings and 196 axial subdivisions because it was intended for long-term depletion (burnup) calculations. However, because the scope of this study relies only on a static, fresh core, these parameters significantly increase computation time while having very little effect on the prediction of k_{eff} . Conversely, the number of axial subdivisions directly dictates the resolution of the CRs. Therefore, reducing the axial subdivisions to 5 drastically reduced computational demand while retaining sufficient resolution to capture the neutron shadowing effect introduced with the insertion of the CRs.

3.3 Data Preprocessing

3.3.1 Data Cleaning and Validating the Monte Carlo Results

The raw dataset produced by the previous stage was processed within a Jupyter notebook to generate a clean input for the surrogate model. First, any samples containing null values for k_{eff} (representing failed OpenMC runs) were removed using the pandas library, ensuring the surrogate model was trained only on the ground-truth results. Compute time was also extracted to establish a baseline for Sec. 4.4. Second, the non-physical parameters (standard deviation and compute time) were excluded from the dataset. A Spearman correlation heatmap (Fig. 3) was generated using `Matplotlib` to visually assess whether the relationships between the parameters in the raw dataset align with physics (Shaqiri *et al.*, 2023).

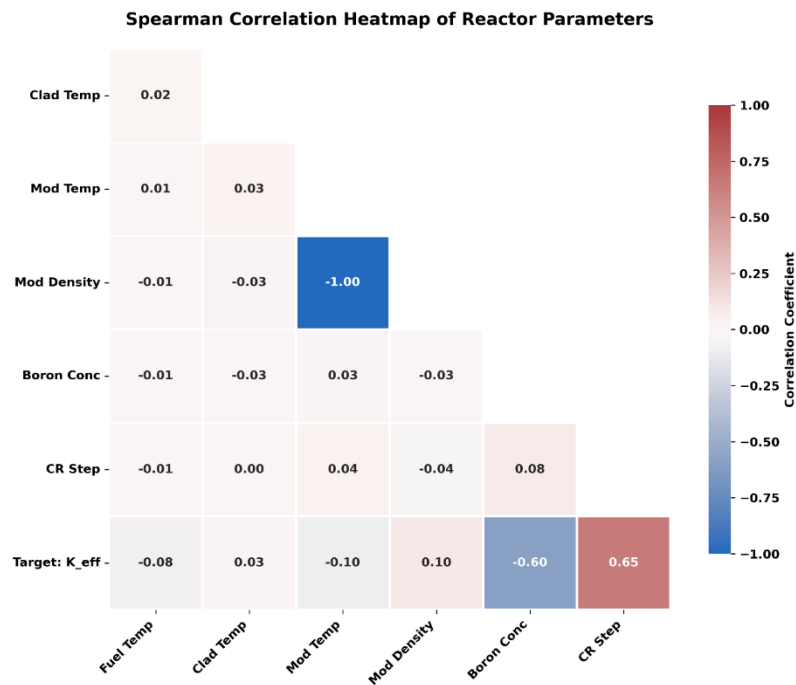


Figure 3: Spearman Correlation Heatmap of Physical Reactor Parameters.

An analysis of the correlation matrix revealed that the interactions between the input features were near zero. This was a strong indication that the Latin Hypercube Sampler worked effectively, generating a range of non-clustered (independent) samples. This eliminated the issue of unintended biases within the random input dataset that can cause overfitting, especially with boosting ML architectures (Breiman, 2001). The strongest correlating features with the target (k_{eff}) were control rod step (0.65) and boron concentration (-0.60). This aligns with the underlying physics that was simulated by the stochastic model, where these features are the primary methods of affecting reactor criticality. Additionally, moderator temperature and density had a

near-perfect correlation (-1.00); this was expected since the MC model uses the IAPWS library to dynamically calculate moderator density based on its temperature with a fixed system pressure. Since moderator temperature and density have a near-perfect correlation, using both as features for the surrogate model has no effect on the accuracy of the model. However, when measuring feature importance, the near-perfect correlation will mask the true importance of these variables (Breiman, 2001), complicating the interpretability as well as adding redundant processing to the ML models. For these reasons, moderator density was excluded from the cleaned dataset alongside the non-physical parameters.

3.4 Surrogate Model Architecture and Training

3.4.1 Tree-Based Algorithm Selection

Due to the computational limitations of generating a high-fidelity dataset, the ML models' sample size was limited to 1 000 (justified in Sec. 3.2.6), which is classified as a small dataset (Tao *et al.*, 2025). Tao *et al.* (2025) explained that overly complex algorithms like NNs are prone to overfitting on small sample sizes due to their excessive complexity. In contrast to this, tree-based learning algorithms like the RFR (Breiman, 2001) reduce overall model bias and variance (overfitting) through the collaboration of multiple ML models, making them far better suited to sparse datasets.

Decision tree architectures are inherently “intuitive and the results are interpretable. Moreover, compared to other algorithms, decision trees require less effort for data preparation, and do not require data normalization and scaling” (Racheal, Liu and Ayodeji, 2022, p. 2). This avoided the NN's major limitation of the “black box dilemma” (Huang *et al.*, 2023, p. 10) by providing transparency through feature importance.

While the RFR served as an effective base model to prove the transparency and speed of tree-based architectures, its predictive accuracy is not as strong as modern ML architectures like eXtreme Gradient Boosting (Chen and Guestrin, 2016). Like the RFR, XGBoost is a tree-based supervised learning algorithm, providing the same transparency and reduction in model bias advantages. However, XGBoost builds trees sequentially, where each new tree learns from the error of previous trees, correcting areas where the previous model underperformed (Bisong, 2019). This boosting architecture makes XGBoost highly effective for achieving a high level of accuracy on complex tabular data (like the physical reactor values).

Both tree-based algorithms were programmed with identical structures (excluding hyperparameters) using the scikit-learn and XGBoost libraries. The training was executed via the scripts `RFR_surrogate.py` and `XGBoost_surrogate.py`, and the resulting fully optimised models were exported using joblib.

3.4.2 Error Metric and Cross-Validation Strategy

Because reactor parameters involve very small fractional deviations, engineers use pcm (percent mille, or parts per hundred thousand) to avoid working with tiny decimals (Mercier, 2025). It is expressed as:

$$pcm = \Delta k_{eff} \times 10^{-5} \quad (3)$$

Both ML training scripts utilised the function `nuclear_scoring_metric` to convert the RMSE to pcm, avoiding tiny decimals when interpreting results uncertainty.

K-fold cross validation is critical when training a limited dataset, to avoid overfitting (Tao *et al.*, 2025) and to provide an accurate prediction of how well the model generalises to an unseen dataset. K-fold validation maximised the usage of the small dataset by ensuring that all samples were used for both training and validation of the model, while avoiding the computational expense of exhaustive resampling methods (Bisong, 2019).

K-fold (with 5 folds) was implemented using scikit-learn for training the ML models by partitioning the dataset into five equal subsamples, training on four and validating on the fifth to ensure that every sample was utilised for one validation. This method ensured that the tree-based ML models did not overfit to the small dataset while also providing a necessary error metric for hyperparameter optimisation.

3.4.3 Hyperparameter Optimisation

RMSE for the ML models was minimised using GridSearch, a hyperparameter tuning module used to optimise model performance by testing the combination of a range of values and picking the best hyperparameters (Bisong, 2019). This ensured that the trade-off between model bias and variance was navigated successfully to prevent overfitting (Bisong, 2019). Bisong (2019) defined model bias as the error introduced by making simplified approximations of complex real-world problems (leading to underfitting), while model variance is the error introduced by the model's sensitivity to noisy data (leading to overfitting).

To optimise both tree-based architectures, two shared hyperparameters were tuned: `n_estimators` and `max_depth`. `n_estimators` dictated the total number of trees/boosting iterations used by each model (Breiman, 2001), while `max_depth` was set to restrict model complexity, ensuring the algorithms do not learn from overly complex interactions that lead to overfitting (Alizadeh, Allen and Mistree, 2020).

For the RFR model specifically, the `max_features` parameter sets the number of input variables selected to determine each node split. This forced the algorithm to learn from a variety of feature combinations rather than just the single dominant parameter (Tao *et al.*, 2025). Additionally, `min_samples_split` set the minimum number of samples required to split a node.

Conversely, the XGBoost algorithm required a different pair of hyperparameters. `learning_rate` scaled the contribution of each newly added tree by a fraction between 0 and 1. A lower learning rate significantly reduces overfitting, though it requires proportionally larger `n_estimators` (Chen *et al.*, 2019). Finally, `subsample` is the fractional ratio of the sample space used to grow each tree. Setting this to a fraction forced the XGBoost to evaluate diverse subsets of the training data, preventing overfitting on the small dataset (Chen *et al.*, 2019).

Table 4: Tested Hyperparameter Values for the 5-fold Cross-validation GridSearch.

ML model	Hyperparameter	GridSearch Hyperparameter Options		
RFR	<code>n_estimators</code>	100	200	300
	<code>max_depth</code>	None	10	20
	<code>max_features</code>	sqrt	log2	1.0
	<code>min_sample_split</code>	2	5	10
XGBoost	<code>n_estimators</code>	1000	1500	2000
	<code>max_depth</code>	3	4	5
	<code>learning_rate</code>	0.01	0.03	0.05
	<code>subsample</code>	0.5	0.65	0.8

Tbl. 4 presents the hyperparameter matrix chosen for each ML model. The GridSearch algorithm found the best combination of these parameters, finding the balance between bias and variance to produce the lowest possible uncertainty (RMSE). These values were chosen using the following strategies:

The RFR relies heavily on the law of Large Numbers, meaning the algorithms generalisation error converges to a stable value as more trees are added (Breiman, 2001). Grid searching up to 300 `n_estimators` ensured that this convergence was reached without redundant computation. Additionally, the error of RFRs is dictated by the correlation between individual trees. Therefore, selecting random `max_features` minimised the correlation between trees, ultimately reducing generalisation error (Breiman, 2001). Including 1.0 allowed the model to use all features to split

trees if it resulted in a reduced RMSE. Limiting `max_depth` and forcing higher `min_sample_split` limited the complexity of the RFR, avoiding overfitting to noise in the limited dataset (Breiman, 2001).

In contrast to the bagging architecture of RFRs, XGBoost uses boosting to reduce bias by learning from the errors of previous trees, making them highly susceptible to overfitting on small datasets (Chen and Guestrin, 2016). By limiting the learning rate to 0.05, the boosting process was forced to be conservative, preventing the model from overfitting to the sparse dataset (Chen *et al.*, 2019). Chen *et al.* (2019) explicitly states that a lower learning rate requires a larger number of estimators; allowing `n_estimators` to reach 2000 allowed the boosting algorithm to account for the reduced learning rate. Furthermore, capping `max_depth` and `subsample` at low values limited tree complexity, ensuring that the XGBoost algorithm did not overfit to the noise in the limited dataset (Bisong, 2019).

3.4.4 Feature Importance and Interpretability

The Gini impurity index is an algorithm used within decision tree architectures that selects the feature (e.g. fuel temperature) that yields the highest “information gain”. It then calculates the feature’s contribution to predicting the target variable; known as Gini impurity, or feature importance (Bisong, 2019). By aggregating how much each feature improves each tree, the Gini index extracted a quantitative feature importance ranking. This ranking could be verified against physics, providing transparency in the ML models’ predictions.

The Gini impurity index was implemented in both the RFR and XGBoost training scripts to provide transparency in how each physical variable drove the model’s prediction of k_{eff} (discussed in Sec. 3.4.1). The resulting Gini importance rankings for each model were extracted to be visually analysed in Sec. 4.3.

3.5 Scope and Assumptions

For the scope of this project, the OpenMC outputs were treated as the absolute ground-truth. However, while extreme high-fidelity benchmarks aim for a standard deviation of 1 to 3 pcm, achieving this requires a massive number of simulated particles (Barrientos and Alfonso, 2021). Because MC uncertainty decreases at a rate of $1/\sqrt{N}$, pushing this uncertainty to such levels would have required exponentially more computation. The accepted mean standard deviation of

75.3 pcm found a balance between computation and predictive accuracy. Consequently, the presence of statistical error in the OpenMC ground-truth inevitably passed into the ML surrogate models as noise, affecting their accuracy.

The ML training dataset was limited to 1 000 samples due to the computational cost of the high-fidelity MC simulations (approximately 60 seconds per sample). While a larger dataset would theoretically improve model accuracy, this sample size provided sufficient diversity within the LHS boundaries to establish a proof-of-concept for tree-based architectures in SMR simulation.

The OpenMC ground-truth dataset was generated using a static solver, so did not consider time-dependent variables like fuel burnup. Therefore, the RFR and XGBoost models could only predict isolated instances of the SMR core. Real-world SMRs are affected by dynamic, time-dependent parameters like fuel depletion/burnup (Mercier, 2025) and delayed neutrons (Zohuri, Balgehshiri and Lomonaco, 2025). However, predicting k_{eff} established the foundational viability of the tree-based models before extending to computationally expensive depletion calculations.

The baseline model relied only on five input features (described in Sec. 3.2.2). While adding more features could theoretically improve model accuracy without significantly increasing prediction time, it introduces diminishing returns as each additional feature had decreasing correlation with the target parameter (k_{eff}). Furthermore, using an excessive number of features diminishes the interpretability of the tree-based models (Alizadeh, Allen and Mistree, 2020), making them harder to audit.

Because the exact design of the NPM is proprietary, some assumptions were made within the MIT ExaSMR benchmark to estimate the restricted designs (described in Sec. 3.2.1). Most importantly, the control rod insertion steps were governed using a benchmark proxy (the BEAVRS benchmark) as suggested by Romano . These assumptions were acceptable for a computational proof of concept, as the fundamental physics of the MC model remained identical to real-world operations. However, deploying an operational model would require a perfect digital twin of the specific SMR.

Although tree-based models are excellent at interpolating predictions, their predictions become far less accurate when extrapolating data (Tao *et al.*, 2025). The scope of the surrogate models in this project was bound by the limits of the LHS; therefore, any reactor states outside the training data could not be accurately predicted. This limitation was mitigated by extending the parameter boundaries beyond what would be expected during standard operation (Described in Sec. 3.2.2).

4 Results

This chapter presents the outcomes of the three-stage pipeline. It critically compares the results of the RFR, XGBoost and Monte Carlo SMR models in terms of baseline validity, predictive accuracy, and computational efficiency.

4.1 Baseline Model Performance

Before evaluating the surrogate models, the fidelity of the OpenMC training data must be verified. The mean MC standard deviation of the baseline dataset was 75.3 pcm . This achieved uncertainty of $< 100\text{ pcm}$ finds the balance between fidelity and computational complexity. The mean standard deviation is very small ($< 0.1\%$), meaning the tree-based models (with sufficient hyperparameter selection) will be able to learn the underlying physics rather than the small amount of noise caused by the MC uncertainty (Breiman, 2001).

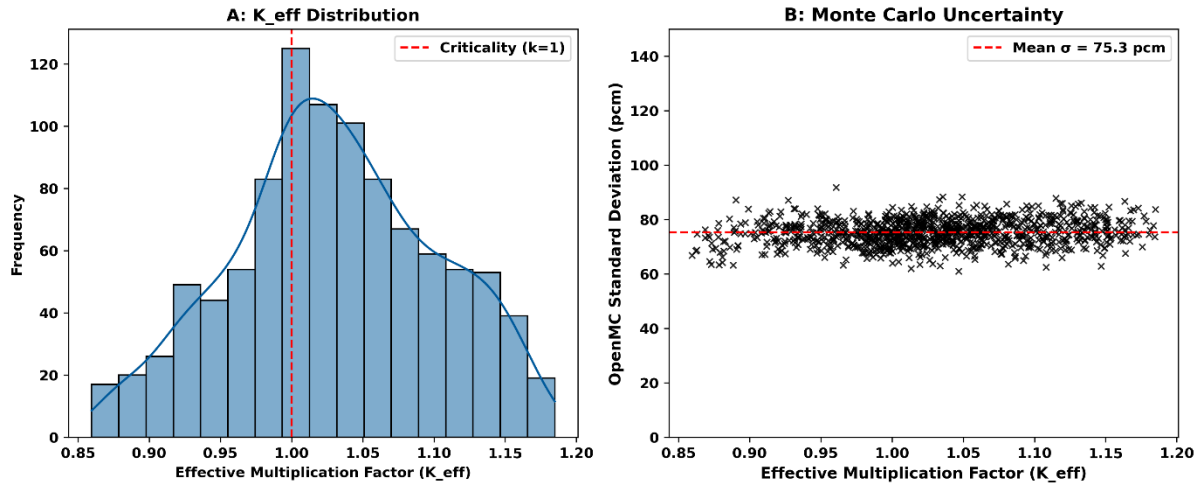


Figure 4: Distribution of the Target Variable and its Corresponding MC Standard Deviation for the Baseline Dataset.

Plot A is a histogram presenting the distribution of k_{eff} across all 1 000 samples. It demonstrates a clear peak near criticality ($k_{eff} = 1$) and spanning a broad range, including both subcritical and supercritical reactor states. This proves that the variations in the input features (Sec. 3.2.2) sufficiently affect criticality to encompass a broad domain of reactor states. Furthermore, the distribution extends past prompt criticality ($k_{eff} = 1 + \beta = 1.0059$, where β is the delayed neutron fraction reported by NuScale Power LLC (2016)). While these extreme combinations of input parameters would be actively avoided during standard operation, their inclusion in the LHS provides the surrogate models with the fundamental physics for extreme reactor states, avoiding the need for extrapolating these values from non-diverse datasets.

Plot B represents a scatter plot of the corresponding MC standard deviations for each of the 1 000 samples/simulations. It forms a flat horizontal cloud centred at a mean of 75.3 *pcm*. Since the statistical uncertainty of MC calculations converges as the number of simulated particles increases (Sec. 2.1), the flatness and lack of outliers in the MC standard deviation confirms that the baseline simulations have reached a stable fidelity across all samples (Gill, Griesheimer and Aumiller, 2017).

4.2 Surrogate Model Performance

To evaluate the performance of the optimised surrogate models, an additional dataset of 200 unseen MC samples was produced using the pipeline described in Ch. 3. Using an unseen test dataset for validating the predictive performance of the surrogate models replicates how they would react in an operational setting. These unseen samples were evaluated using the saved ML models, which used the optimised GridSearch hyperparameters detailed in Tbl. 5.

Table 5: Best GridSearch Hyperparameters and RMSE for the ML Surrogate Models.

ML model	Hyperparameter	Best Hyperparameter	RMSE (pcm)
RFR	n_estimators	200	779.7
	max_depth	None	
	max_features	1	
	min_sample_split	2	
XGBoost	n_estimators	2000	336.8
	max_depth	3	
	learning_rate	0.03	
	subsample	0.5	

Tbl. 5 demonstrates that the XGBoost model achieved an RMSE of 336.8 *pcm*, outperforming the RFR (with a recorded RMSE of 779.7 *pcm*) model by more than a factor of two. This reduction in error highlights XGBoost's superior capability of handling the complex, non-linear physics involved in nuclear reactor simulation. Despite being outmatched by the boosting algorithm, the RFR's RMSE of < 1 000 *pcm* (which is less than 1%) confirms that its foundational tree-based algorithm successfully captured the underlying physics, proving it as a viable baseline surrogate model. However, it is important to note that these RMSE values are extracted using k-fold cross-validation, so a more accurate representation of model uncertainty is visualised in Fig. 5 and 6 with the unseen 200-sample dataset.

OpenMC Ground Truth vs. Surrogate Model Predictions

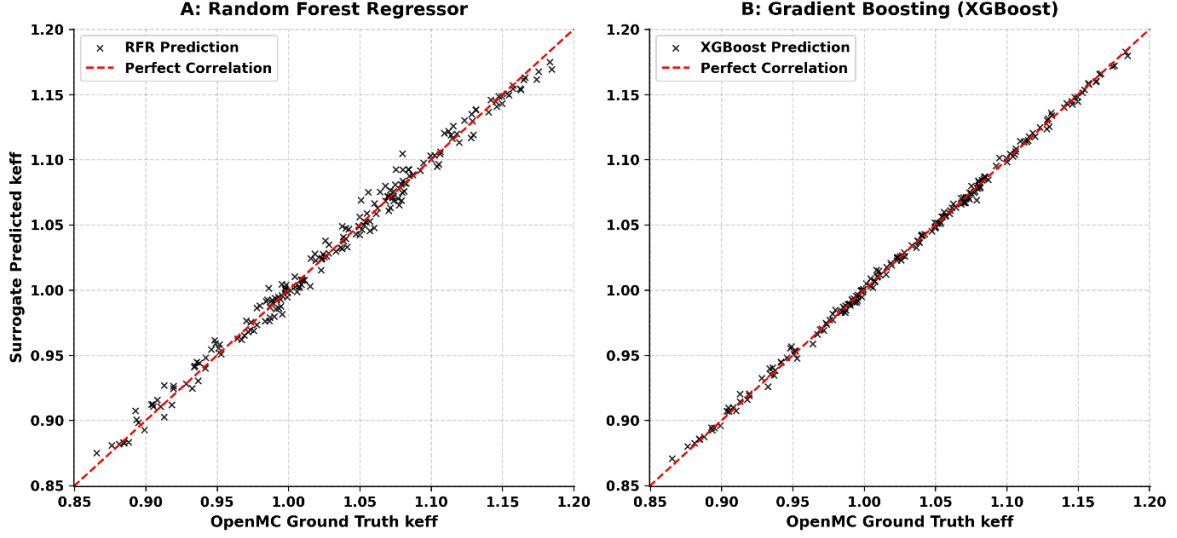


Figure 5: Parity Plots Evaluating the Predictive Performance of the Surrogate Models against the MC Ground-Truth.

Fig. 5 shows the deviations in predicted k_{eff} of the ML models against the MC high-fidelity model with the unseen dataset. It reveals that the XGBoost predictions are tightly distributed around the perfect correlation line, uniformly across all samples. In contrast, the RFR parity plot has a wider spread, becoming more severe at the extreme subcritical and supercritical ends. This indicates that the bagging architecture of the RFR provides a stable correlation where k_{eff} distribution is dense, but the sparsity of the data at the extremes (seen in Fig. 4) has caused underfitting in the RFR model. The more modern XGBoost model overcomes this with its sparsity-aware boosting architecture (Chen and Guestrin, 2016, Sec. 3.4), which has managed to maintain high accuracy even at the extreme edges of the sample range. Additionally, the fact that both models fit relatively tightly (without outliers) suggests that the GridSearch algorithm successfully converged to produce a stable RMSE, meaning that further significant increase in fidelity would require a larger dataset rather than further hyperparameter tuning. Furthermore, this tight correlation on the unseen test data answers the question raised in Sec. 2.4. Despite literature warnings that boosting algorithms may be susceptible to noise in sparse datasets (Alizadeh, Allen and Mistree, 2020), the XGBoost surrogate model demonstrated no signs of overfitting, proving its robustness on sparse nuclear datasets.

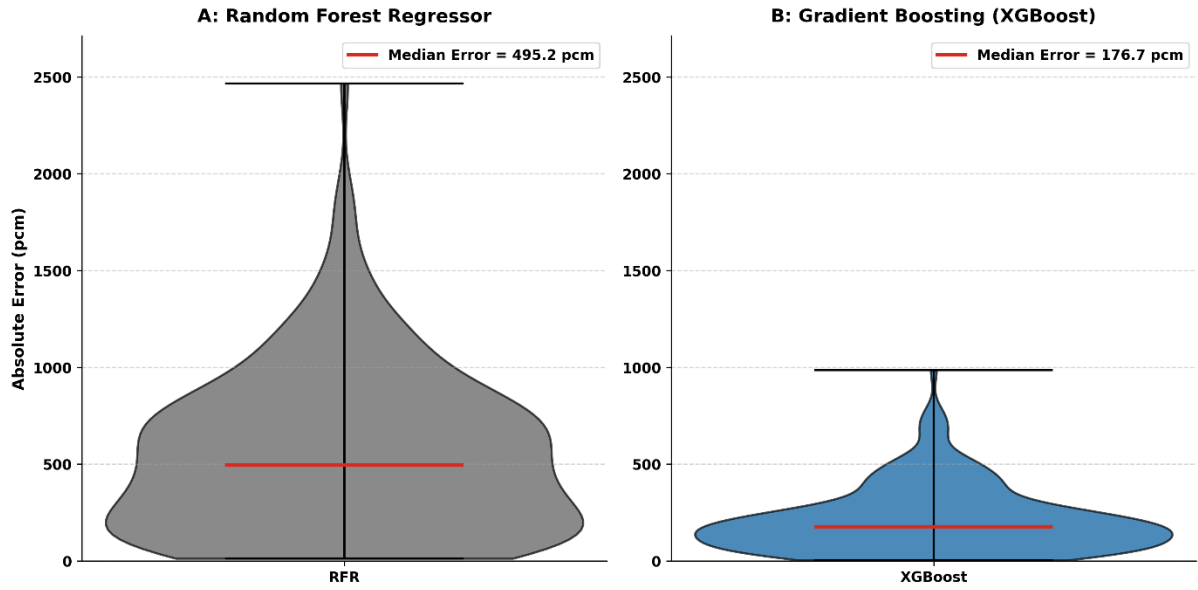


Figure 6: Violin Plot Showing the Probability Density of Error in the ML Model Predictions.

Finally, Fig. 6 visualises the probability density of error for each prediction (of the unseen dataset). The RFR architecture yields a median error of 495.2 *pcm*, evident in its bottom-heavy bulge. However, a small portion of outliers reach approximately 2500 *pcm* uncertainty. Conversely, the XGBoost architecture yields a significantly lower median error of 176.7 *pcm*, with a more compact bottom-heavy bulge. XGBoost outliers reach 1 000 *pcm* while a significant number of the RFR's predictions are above this threshold. The bottom-heavy shape of both violin plots verifies that the majority of predictions made by both tree-based models are achieving a stable fidelity. Ultimately, the above observations of Fig. 6 validate that the gradient boosting architecture of XGBoost makes complex physics-based predictions far more effectively than the RFR model in terms of predictive accuracy.

4.3 Surrogate Model Interpretability

To provide a transparency improvement over NN-based approaches, the tree-based surrogate models used the Gini impurity index (described in Sec. 3.4.4) to extract feature importance. Evaluating these rankings allows for the verification of whether the ML architectures successfully learned the underlying physics within the SMR. Fig. 7 uses the Gini impurity index to rank the feature importance for both ML models.

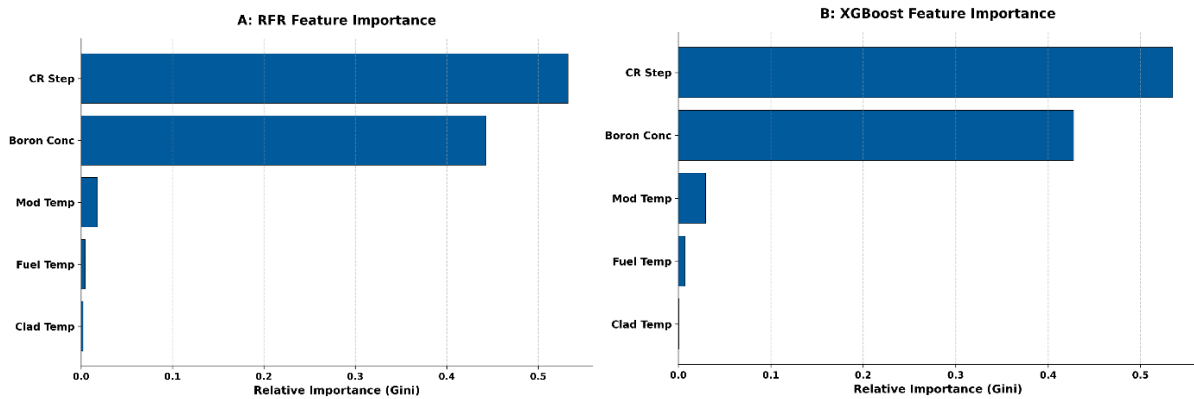


Figure 7: Gini Feature Importance Ranking of Reactor Input Parameters for RFR and XGBoost Surrogate Models.

Fig. 7 shows that both architectures identify control rod step and boron concentration as the most influential variables in predicting k_{eff} . This aligns with physics because CRs and boron are the two primary mechanisms for controlling reactivity (NuScale Power LLC, 2016). Interestingly, the Gini importance of fuel temperature is dwarfed by the two primary features; this can be explained by the neutron absorbing effects of the Doppler effect (Mercier, 2025) being overshadowed by the effects of the two primary neutron absorbing mechanisms. Furthermore, these observations match the statistical correlation from the Spearman correlation matrix (Sec. 3.3.1, Fig. 3). This proves that both models learn the physical hierarchy of features, providing a level of transparency for validation.

Additionally, Fig. 7 shows a slight difference between the RFR and XGBoost models. This is explained by the inherent differences between bagging and boosting architectures being inflated due to having a small dataset. RFRs use the `max_features` hyperparameter to force the algorithm to learn from a variety of feature combinations (Tao *et al.*, 2025), whereas XGBoost concentrates feature importance on the dominant drivers (Chen and Guestrin, 2016). This observation is a direct result of using a limited dataset and likely makes the RFR more accurate for the less dominant features and XGBoost more accurate for the dominant features in this case.

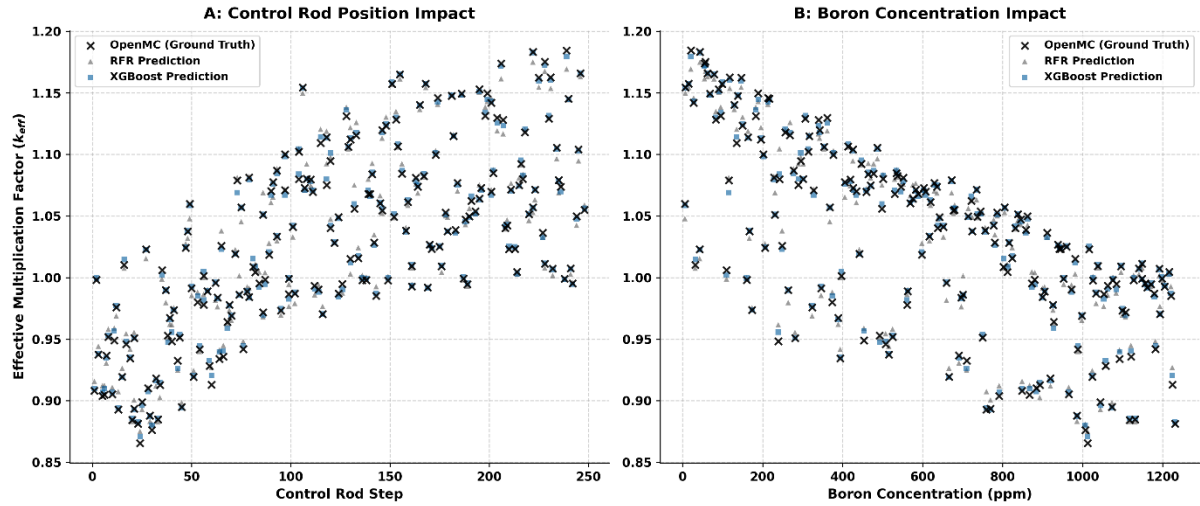


Figure 8: Surrogate Model Predictions vs. Monte Carlo Ground-Truth for Two Dominant Features.

Fig. 8 shows the prediction of k_{eff} across the entire ranges of the two dominant features for all three reactor models. They were generated using predictions of the unseen dataset described in Sec. 4.2, replicating how the models would react to operational data. As evidenced in Fig. 8, both surrogate models effectively map the complex non-linear relationship between CR step/boron concentration and k_{eff} , as shown by the ground-truth OpenMC predictions. Slight deviations between surrogate model predictions and the ground-truth are consistent throughout the feature range and can be explained by RMSE discussed in Sec. 4.2. Additionally, observations in Sec. 4.2 match the behaviour seen in Fig. 8 where the XGBoost's error is smaller than the RFR's by approximately a factor of two.

4.4 Computational Speed-Up

Beyond transparency, the viability of tree-based surrogate models for real-world usage in predicting SMR parameters requires an evaluation of the surrogate models against the baseline MC model not just on predictive accuracy, but on their ability to resolve the computational bottleneck present in MC methods.

Table 6: Summary of Predictive Uncertainty and Computational Speed-Up for the Evaluated Reactor Models.

Performance Metric	Monte Carlo (Ground-Truth)	RFR Surrogate Model	XGBoost Surrogate Model
Mean Uncertainty (pcm)	75.3	779.7	336.8
Median Uncertainty (pcm)	75.4	495.2	176.7
Training/Generation Time (s)	60 468.11 (16.80 hours)	22.02	29.28

Avg. Prediction Time (s)	60.47	0.000070	0.000024
Relative Speed-Up Factor	1x (Baseline)	864 000x	2 519 000x

As demonstrated in Tbl. 6, the generation of the high-fidelity MC data imposes a severe computational burden of 16.8 hours for 1 000 samples. However, once this baseline data is generated, both tree-based models require less than 30 seconds to train. For each simulation, the stochastic OpenMC model requires an average of 60.47 seconds of computation, whereas the ML surrogate models reduce this inference time to an order of microseconds. This shift in computational burden from individual simulations to one-time training generation results in massive relative speed-up factors.

The high computational speed of tree-based ML models over MC methods comes with the trade-off of predictive accuracy. Although the RFR achieved a significant speed-up factor of 864 000x with an uncertainty of 779.7 *pcm*, the modernised architecture of XGBoost definitively improves this to reduce predictive uncertainty to a more acceptable level of 336.8 *pcm*, while reducing prediction time by a factor of 2 519 000x. Since further improvements to predictive accuracy would require a larger training dataset (established in Sec. 4.2), XGBoost proves itself as a highly viable model for reactor parameter prediction that significantly balances the trade-off between speed and the computational demand required by traditional models.

5 Discussion

5.1 Interpretation of Results

The XGBoost model achieved a mean and median predictive uncertainty of 336.8 and 176.7 *pcm*, respectively. While this is two to three times higher than the 100 *pcm* benchmark established in current literature (discussed in Sec. 5.2), this result is dependent on the context of data scarcity. The deliberate use of a limited 1 000-sample dataset was designed to test the validity of tree-based architectures without incurring excessive computational costs. As demonstrated by Oktavian (2023), achieving sub-100 *pcm* uncertainty requires more than double the number of samples; their XGBoost model predicted k_{eff} with an average error of 50 *pcm* for 2 400 samples and 40 *pcm* for 4 800 samples. Therefore, the achieved accuracy of 336.8 *pcm* successfully demonstrates that XGBoost can capture the underlying physics even on a limited dataset,

establishing a foundation for scaling the model to a benchmark level of performance when provided with an expanded training dataset.

The tree-based surrogate models successfully map the non-linear relationships between input variables and k_{eff} using up to 2.5 million times less computation than MC methods. While the generation of high-fidelity MC training data imposes an initial computational burden, this should be viewed as a strategic one-time computational investment. Improving the fidelity of the surrogate models would require more simulated particles (discussed in Sec. 3.5) and a larger dataset, both of which inherently increase the computational cost of the ML training dataset. However, after this initial sunk cost, it unlocks a permanent model with the capability for microsecond inference times. Ultimately, the methodology validates that the high upfront cost of MC data generation is effectively offset by the speed and scalability of the resulting ML surrogate model.

5.2 Comparison Against Existing Studies

The literature heavily targets an error of 100 *pcm* as a benchmark for accurate predictive models (established in Sec. 2.2). Neural network-based surrogate models, such as the Back Propagation Neural Network (BPNN) used by Bei *et al.* (2023), have successfully achieved this level of accuracy on larger datasets. However, the highly complex internal parameters inherent in NNs render them fully opaque “black boxes” (Huang *et al.*, 2023). In contrast, tree-based ML models are far less complex, providing an additional level of transparency and robustness that NNs inherently lack.

Beyond interpretability, the literature reveals a significant difference in the computational scalability between the tree and NN-based architectures. The BPNN utilised by Bei *et al.* (2023) achieved an average prediction time of 2 milliseconds. When compared against the 24-microsecond inference time achieved by the XGBoost model in this study, the tree-based architecture proves to be two orders of magnitude faster than a comparable NN. This suggests that the simpler, tree-based algorithms are superior for large-scale SMR simulations where millions of inferences are required.

Furthermore, while literature confirms that tree-based architectures have reached sub-100 *pcm* accuracy, Oktavian *et al.* (2024) noted that Deep Neural Networks (DNNs) can still slightly outperform XGBoost by approximately 30% in predictive accuracy. However, in the context of nuclear engineering, this small gain in accuracy comes at the cost of interpretability. Therefore,

tree-based models offer a verifiable alternative to NNs, representing an engineering compromise between predictive accuracy and model transparency.

5.3 Regulatory and Safety Implications

Gini impurity successfully proves that the surrogate models learned the underlying physical hierarchy of features, providing a vital level of confidence in their validity. Despite this, XGBoost models are still widely classified as black boxes (Chen *et al.*, 2019). While a small number of shallow decision trees is easily interpretable, RFR and XGBoost rely on the aggregation of hundreds or thousands of trees respectively. This architecture inherently obscures the underlying decision-making process, rendering the interpretability of the tree-based algorithms highly “questionable” (Alizadeh, Allen and Mistree, 2020, p. 284).

From a regulatory standpoint, the success of predictive accuracy and computational efficiency, demonstrated by the surrogate models on a limited dataset, indicates that tree-based algorithms are a viable solution to the computational bottleneck. However, ALARP principles explicitly mandate “robust, transparent and accountable” AI models (Huang *et al.*, 2023, p. 10). While feature importance proves statistical association, the current surrogate models lack “accountability” due to the obscurity of their decision-making pathways. To achieve ALARP compliance, an SMR surrogate model must provide explicit explainability rather than just feature importance (Office for Nuclear Regulation, CNSC and USNRC, 2024).

Because an SMR operator cannot accountably justify real-time predictions, tree-based surrogate models cannot currently be legally deployed within control or safety systems for SMRs. Therefore, the XGBoost surrogate model is best suited for preliminary designs and studies with rapid timelines where computational speed is prioritised over extreme fidelity. For safety and licensing applications, the industry must still rely on traditional MC validation to achieve the extreme high-fidelity and verifiability mandated by the ALARP principles (Office for Nuclear Regulation, CNSC and USNRC, 2024).

6 Reflective Statement

6.1 Societal and Environmental Impact

The deployment of SMRs represents a pathway toward achieving the UK's commitment to Net Zero by 2050 (*The Climate Change Act 2008 (2050 Target Amendment)*, 2019). They offer a safe, factory-manufactured, low-carbon alternative for generating power compared to fossil fuel infrastructure. Additionally, the UK aims to deploy up to 24GW of SMR capacity by 2050 (Department for Energy Security and Net Zero, 2024). However, the time-consuming, computationally expensive design cycle of this underutilised technology acts as a significant delay to widespread deployment. The primary capability of the three-stage pipeline created in this study is an ML model that reduces the computation time of traditional MC methods by 2 500 000 times. By successfully training a surrogate model to predict k_{eff} , this research demonstrates that the computational bottleneck in SMR simulations can be bypassed; consequently, allowing nuclear engineers to accelerate the timeframe of SMR deployment.

Conversely, the primary adverse impact of this project is the carbon emissions generated during the computationally intensive creation of the baseline dataset (using MC methods). In order to create a high-fidelity training dataset for the ML models, the OpenMC simulations required 16.8 hours of continuous computation. Utilising a Lenovo YOGA Slim 7i Pro drawing a maximum power of 30 W, the total energy consumption for this phase was approximately 0.504 kWh. Applying the UK National Grid's average carbon intensity of 124 gCO₂e/kWh (NESO *et al.*, 2026), the embodied carbon footprint of the baseline dataset (lower bound) is 0.062kg CO₂e. However, when compared with MC methods, this one-time upfront cost is quickly offset by the computational efficiency provided by the saved tree-based surrogate model. Thus, the ML pipeline not only provides a solution to the computational bottleneck but presents a far more energy-efficient and sustainable method for future simulations of nuclear reactors.

6.2 Ethics and Safety

As this project completely relied on simulated physics data (MIT ExaSMR benchmark and ENDF/B-VII.1 libraries), it did not involve human participants or data. Therefore, ethical approval was not required for this study. Additionally, all software and nuclear data (ENDF/B-VII.1) in this study are publicly available, ensuring reproducibility and transparency in the methodology.

Although this study is purely computational, it has safety implications if utilised in simulations linked to the operation of a nuclear reactor. As discussed in Sec. 5.3, the tree-based models provide transparency but fall short of the accountability and explainability required for fully satisfying the regulatory ALARP principles. Therefore, it is important to establish that these models are appropriate for preliminary designs, but not for informing safety-critical systems.

To ensure computational safety, the methodology incorporated multiple verification steps ensuring that each part of the pipeline operates as intended. Examples of this are: error message outputs for each major point of error in the code, verifying the correlation heatmap (Fig. 3) against nuclear physics, validating Gini importance (Fig. 7) against the correlation heatmap, and testing the ML model on an unseen LHS set of samples.

7 Conclusions

7.1 Conclusion

This project successfully developed and evaluated two interpretable tree-based ML surrogate models capable of predicting the criticality (k_{eff}) of a Small Modular Reactor (SMR), thereby reducing the computational demand required for reactor simulations. The XGBoost surrogate model achieved an average uncertainty of 336.8 *pcm* with a limited dataset, combined with more than a 2 500 000x computational speed-up for predicting k_{eff} . These results serve as a proof-of-concept that tree-based ML models are well-suited to learning the complex non-linear correlations between physical reactor parameters in an SMR, while fundamentally resolving the computational bottleneck required by traditional MC methods.

The methodology successfully delivered on all objectives set out in Sec. 1.2: a high-fidelity OpenMC baseline model with an average uncertainty of 75.3 *pcm*, a robust data-preprocessing pipeline, and optimised tree-based models achieving microsecond prediction times. However, the critical evaluation revealed a significant limitation in the viability of tree-based surrogates for satisfying the regulatory requirements set out by the ONR. Despite their inherent transparency, tree-based ML models lack the explicit explainability specifically mandated for reactor simulations pertaining to operation and safety (under ALARP). Therefore, the appropriate use case for tree-based SMR surrogate models in the current state of the industry is for preliminary designs and studies with rapid timelines, where a higher predictive uncertainty is an acceptable trade-off for reduced computation time.

This study directly addresses the computational and regulatory barriers hindering high-fidelity SMR deployment identified in Sec. 1.1. By demonstrating that tree-based models can capture the complex physical correlations within an SMR, this research bridges the gap identified in Sec. 2.4: the lack of transparent ML architectures applied to SMR simulations. Furthermore, the massive reduction in computation achieved by XGBoost presents a viable pathway toward accelerating the deployment of SMRs, supported by the UK's commitment to 24GW of nuclear power capacity by 2050 (Department for Energy Security and Net Zero, 2024). While current tree-based ML models fall short of ALARP compliance for safety-critical applications, they establish a transparent alternative to fully opaque neural networks. This represents a necessary step towards improving the explainability and accountability of AI in nuclear engineering.

7.2 Future Work

Expanding the Dataset:

To achieve benchmark performance (< 100 pcm uncertainty), future work must prioritise expanding the training dataset to between 2 000 and 5 000 samples. Oktavian (2023) demonstrated this with an XGBoost model that predicts k_{eff} with an average of 50 pcm error for 2 400 samples and 40 pcm error for 4 800 samples. A quantitative analysis should be conducted to map the computation-accuracy trade-off between dataset size and model fidelity.

Additionally, a key limitation of this study (discussed in Sec. 3.5) was that the baseline OpenMC model only simulated static reactor states using five input features. Future work should transition to dynamic models by incorporating time-dependent variables like fuel burnup, depletion and delayed neutrons (Zohuri, Balgehshiri and Lomonaco, 2025). Demonstrating the viability of this approach, Hashemi, Macián-Juan and Ohlerich (2026) successfully applied ML to reduce the ENDF/B-VII.1 nuclear cross-section library for dynamic simulations of a commercial PWR (the Russian VVER-1000) over a one-year period, improving computation time by around 40%. Integrating these time-dependent parameters into a tree-based surrogate model would allow engineers to benefit from their microsecond inference times for complex reactor studies. However, a study should be conducted to ensure this added complexity does not severely limit the interpretability of the ML model (Alizadeh, Allen and Mistree, 2020).

Breaking the Regulatory Barrier:

To align with strict ONR regulations, future implementations should move beyond basic feature importance and integrate Explainable Artificial Intelligence (XAI) techniques. Some current

implementations of this are Shapley Additive Explanations (SHAP) and Local Interpretable Model-agnostic Explanations (LIME) which are identified by Huang *et al.* (2023) to be a promising method of providing the “accountability” required for satisfying the ALARP principle. In addition to incorporating explainability for an ALARP-compliant tree-based surrogate, future work should use an exact twin OpenMC SMR model (this study used a model with assumptions discussed in Sec. 3.2.1) to ensure the surrogate reflects how the SMR behaves in real-world operation.

Alternative Approaches:

Alternative methodologies include transferring the learning from pre-existing reactor datasets to reduce the demand for high-cost MC simulations; Tao *et al.* (2025) identified “transfer learning” as a powerful tool, although future work should evaluate the benefits of this for nuclear specific applications.

Additionally, while hybrid ML-physics approaches have been successfully used by Oktavian (2023) to increase the fidelity of ML predictions using deterministic methods, the trilateral AI principles paper by the Office for Nuclear Regulation, CNSC and USNRC (2024) highlights that hybrid approaches could be utilised to benefit from the explainability of physics informed methods while still reducing computational demand.

The Next Step:

The immediate priority for advancing this study is expanding the static training dataset to beyond 2000 samples to reach a fidelity of less than 100 *pcm*, while simultaneously integrating XAI through XGBoost’s native SHAP integration (Chen *et al.*, 2019). This aims to transition the surrogate model from offering basic interpretability to delivering the explainability required by ALARP.

Bibliography

Alizadeh, R., Allen, J. K. and Mistree, F. (2020) 'Managing computational complexity using surrogate models: a critical review', *Research in Engineering Design*, 31(3), pp. 275–298. Available at: <https://doi.org/10.1007/s00163-020-00336-7> (Accessed: 1 April 2026).

Anthropic (2026) Claude (Sonnet 4.5 Extended) [Large language model]. Available at: <https://claude.ai> (Accessed: 1 April 2026).

Barrientos, R. and Alfonso, J. (2021) 'Time-dependent Monte Carlo in fissile systems with beta-delayed neutron precursors', pp. Available at: <https://arxiv.org/abs/2101.09338> (Accessed: 1 April 2026).

Basu, K., Hao, J., Hintz, D., Shah, D., Palmer, A., Hora, G. S., Nwankwo, D. and White, L. (2022) 'Uncertainty quantification methods for ML-based surrogate models of scientific applications', *ML for Physical Sciences*, NeurIPS, 2022, pp. Available at: https://raw.githubusercontent.com/ml4physicalsciences/ml4physicalsciences.github.io/master/2022/files/NeurIPS_ML4PS_2022_183.pdf (Accessed: 1 April 2026).

Bei, X., Dai, Y., Yu, K. and Cheng, M. (2023) 'Three-Dimensional Surrogate Model Based on Back-Propagation Neural Network for Key Neutronics Parameters Prediction in Molten Salt Reactor', *Energies*, 16(10), pp. Available at: <http://dx.doi.org/10.3390/en16104044> (Accessed: 1 April 2026).

Bisong, E. (2019) 'Introduction to Scikit-learn', *Building machine learning and deep learning models on google cloud platform: a comprehensive guide for beginners*. ed.: Springer, pp. 215–229. Available at: <https://link.springer.com/content/pdf/10.1007/978-1-4842-4470-8.pdf> (Accessed: 1 April 2026).

Breiman, L. (2001) 'Random Forests', *Machine Learning*, 45(1), pp. 5–32. Available at: <https://doi.org/10.1023/A:1010933404324> (Accessed: 1 April 2026).

Chadwick, M. B., Herman, M., Obložinský, P., Dunn, M. E., Danon, Y., Kahler, A. C., Smith, D. L., Pritychenko, B., Arbanas, G., Arcilla, R. and Brewer, R. (2011) 'ENDF/B-VII. 1 nuclear data for science and technology: cross sections, covariances, fission product yields and decay data', *Nuclear Data Sheets*, 112(12), pp. 2887–2996. Available at: <https://www.sciencedirect.com/science/article/pii/S009037521100113X> (Accessed: 1 April 2026).

Chen, T. and Guestrin, C. 'Xgboost: A scalable tree boosting system'. *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*, 2016. 785–794. Available at: <https://dl.acm.org/doi/epdf/10.1145/2939672.2939785> (Accessed: 1 April 2026).

Chen, T., Tong, H., Michael, B. and Vadim, K. (2019) 'Package 'xgboost'', *R version*, 90(1-66), pp. 40. Available at: <https://deepsense.ai/wp-content/uploads/2017/03/xgboost.pdf> (Accessed: 1 April 2026).

The Climate Change Act 2008 (2050 Target Amendment) (Order 2019, SI 2019/1056,).

Department for Energy Security and Net Zero (2024) *Civil Nuclear: Roadmap to 2050*. London: HM Government (CP 1009). Available at: <https://www.gov.uk/government/publications/civil-nuclear-roadmap-to-2050> (Accessed: 1 April 2026).

Devan, P. and Khare, N. (2020) 'An efficient XGBoost–DNN-based classification model for network intrusion detection system', *Neural Computing and Applications*, 32(16), pp. 12499–12514. Available at: <https://link.springer.com/article/10.1007/s00521-020-04708-x> (Accessed: 1 April 2026).

Gill, D. F., Griesheimer, D. P. and Aumiller, D. L. (2017) 'Numerical Methods in Coupled Monte Carlo and Thermal-Hydraulic Calculations', *Nuclear Science and Engineering*, 185(1), pp. 194–205. Available at: <https://doi.org/10.13182/NSE16-3> (Accessed: 1 April 2026).

Gong, H., Cheng, S., Chen, Z. and Li, Q. (2022) 'Data-Enabled Physics-Informed Machine Learning for Reduced-Order Modeling Digital Twin: Application to Nuclear Reactor Physics', *Nuclear Science and Engineering*, 196(6), pp. 668–693. Available at: <https://doi.org/10.1080/00295639.2021.2014752> (Accessed: 1 April 2026).

Hall, A. and Agarwal, V. (2024) 'Barriers to adopting artificial intelligence and machine learning technologies in nuclear power', *Progress in Nuclear Energy*, 175, pp. 105295. Available at: <https://www.sciencedirect.com/science/article/pii/S0149197024002452> (Accessed: 1 April 2026).

Harris, C. R., Millman, K. J., Van Der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J. and Kern, R. (2020) 'Array programming with NumPy', *nature*, 585(7825), pp. 357–362. Available at: <https://www.nature.com/articles/s41586-020-2649-2> (Accessed: 1 April 2026).

Hashemi, A., Macián-Juan, R. and Ohlerich, M. (2026) 'Evaluating machine learned nuclear data precision in full core nuclear reactor Monte Carlo neutronics and computational efficiency analyses', *Scientific Reports*, 16(1), pp. 1314. Available at: <https://doi.org/10.1038/s41598-026-35227-9> (Accessed: 1 April 2026).

Horelik, N., Herman, B., Forget, B. and Smith, K. 'Benchmark for evaluation and validation of reactor simulations (BEAVRS), v1. 0.1'. *Proc. Int. Conf. Mathematics and Computational Methods Applied to Nuc. Sci. & Eng.*, 11 April 2018. 63–68. Available at: https://bpb-us-e1.wpmucdn.com/sites.mit.edu/dist/9/643/files/2017/01/BEAVRS_2.0.2_spec.pdf (Accessed: 1 April 2026).

Huang, Q., Peng, S., Deng, J., Zeng, H., Zhang, Z., Liu, Y. and Yuan, P. (2023) 'A review of the application of artificial intelligence to nuclear reactors: Where we are and what's next', *Heliyon*, 9(3), pp. Available at: <https://doi.org/10.1016/j.heliyon.2023.e13883> (Accessed: 10 March 2026).

International Atomic Energy Agency (2007) *Deterministic analysis of operational events in nuclear power plants*, Vienna: International Atomic Energy Agency (IAEA-TECDOC-1550. Available at: https://www-pub.iaea.org/MTCD/Publications/PDF/TE_1550_web.pdf (Accessed: 19 March 2026).

Kecek A, Tucek K, Holmstrom B and U.P, V. (2016) *Development of M5 cladding material correlations in the TRANSURANUS code: revision 1*. Available at: <https://publications.jrc.ec.europa.eu/repository/handle/JRC100644> (Accessed: 1 April 2026).

Kretzschmar, H. J. and Wagner, W. (2019) 'IAPWS industrial formulation 1997 for the thermodynamic properties of water and steam', *International Steam Tables: Properties of Water and Steam Based on the Industrial Formulation IAPWS-IF97*. ed.: Springer, pp. 7–150. Available at: https://link.springer.com/chapter/10.1007/978-3-662-53219-5_3 (Accessed: 1 April 2026).

McKinney, W. (2010) 'Data structures for statistical computing in Python', *Proceedings of the 9th Python in Science Conference*, 445(1), pp. 51–56. Available at: https://pdfs.semanticscholar.org/ef4e/f7f38bb907e5d7b4df3e6ff1db269d4970f5.pdf?_gl=1*1t91t2v*_ga*OTE5NDAXODkxLjE3MTcyMDIyOTM.*_ga_H7P4ZT52H5*MTcxNzlwMjl5My4xLjAuMTcxNzlwMjMwMy41MC4wLjA (Accessed: 1 April 2026).

Mercier, B. (2025) *Simple Models for Understanding Nuclear Reactor Physics*. Springer Science+Business Media, p. Reprint. Available at: <https://link.springer.com/content/pdf/10.1007/978-3-031-98532-4.pdf> (Accessed: 1 April 2026).

NESO, EDF, University of Oxford and WWF (2026) 'Carbon Intensity API', pp. Available at: <https://carbonintensity.org.uk/> (Accessed: 1 April 2026).

NuScale Power LLC (2016) *NuScale standard plant design certification application, part 2 - tier 2, chapter 4: reactor (Revision 0)*: NuScale Power LLC. Available at: <https://www.nrc.gov/docs/ML1701/ML17013A274.pdf> (Accessed: 31 March 2026).

Office for Nuclear Regulation (2020) *Safety Assessment Principles for Nuclear Facilities (2014 Edition, Revision 1)*: ONR. Available at: <https://www.onr.org.uk/media/pobf24xm/saps2014.pdf> (Accessed: 1 April 2026).

Office for Nuclear Regulation (2025) *Validation of Computer Codes and Calculation Methods (NS-TAST-GD-042)*: ONR (NS-TAST-GD-042). Available at: <https://www.onr.org.uk/media/rv5fzsc2/ns-tast-gd-042.docx> (Accessed: 1 April 2026).

Office for Nuclear Regulation, CNSC and USNRC (2024) *Considerations for Developing Artificial Intelligence Systems in Nuclear Applications*: ONR. Available at: https://www.onr.org.uk/media/03zl1osf/canukus_trilateral_ai_principles_paper_2024_08_28-final.pdf (Accessed: 1 April 2026).

Oktavian, M. R. (2023) *Development of a Machine Learning-Assisted Core Simulation for Boiling Water Reactor Operations*. ProQuest LLC [Online] Available at: <https://www.proquest.com/docview/3122640411?fromopenview=true&pq-origsite=gscholar&sourcetype=Dissertations%20%20Theses> (Accessed: 1 April 2026).

Oktavian, M. R., Nistor, J., Gruenwald, J. T. and Xu, Y. (2024) 'Integrating core physics and machine learning for improved parameter prediction in boiling water reactor operations', *Scientific Reports*, 14(1), pp. 5835. Available at: <https://doi.org/10.1038/s41598-024-56388-5> (Accessed: 1 April 2026).

OpenMC Development Team (2024) OpenMC Python API Documentation. Available at: <https://docs.openmc.org/en/stable/Pythonapi/index.html> (Accessed: 1 April 2026).

Racheal, S., Liu, Y. and Ayodeji, A. (2022) 'Evaluation of optimized machine learning models for nuclear reactor accident prediction', *Progress in Nuclear Energy*, 149, pp. 104263. Available at: <https://link.springer.com/article/10.1007/s41365-026-01928-z> (Accessed: 1 April 2026).

Reyes, J. N. J. and Ingersoll, D. T. (1996) 'NuScale Power Small Modular Reactor', *Nuclear Espana*, pp. 14–17. Available at: <https://www.revistanuclear.es/wp-content/uploads/hemeroteca/380/NE380-02.pdf> (Accessed: 1 April 2026).

Robertson, R. C. (1965) *MSRE Design & Operations Report Part 1 Description of Reactor Design*, United States. Available at: <https://www.osti.gov/servlets/purl/4654707> (Accessed: 1 April 2026).

Romano, P. K. ExaSMR Benchmark. Available at: <https://github.com/mit-crpg/ecp-benchmarks> (Accessed: 1 April 2026).

Romano, P. K., Horelik, N. E., Herman, B. R., Nelson, A. G., Forget, B. and Smith, K. (2015) 'OpenMC: A state-of-the-art Monte Carlo code for research and development', *Annals of Nuclear Energy*, 82, pp. 90–97. Available at: <https://www.sciencedirect.com/science/article/pii/S030645491400379X> (Accessed: 1 April 2026).

Sanchez, P. P. and dos Santos, A. (2021) 'Prediction of the Power Peaking Factor in a Boron-Free Small Modular Reactor Based on a Support Vector Regression Model and Control Rod Bank Positions', *Nuclear Science and Engineering*, 195(5), pp. 555–562. Available at: <https://doi.org/10.1080/00295639.2020.1854541> (Accessed: 1 April 2026).

Shaqiri, M., Iljazi, T., Kamberi, L. and Ramani-Halili, R. (2023) 'Differences between the correlation coefficients Pearson, Kendall and Spearman', *Journal of Natural Sciences and Mathematics of UT*, 8(15–16), pp. 392–397. Available at: <https://eprints.unite.edu.mk/1557/1/JNSM%202023-392-397.pdf> (Accessed: 1 April 2026).

Subki, H. (2020) *Advances in Small Modular Reactor Technology Developments*, Vienna, Austria: International Atomic Energy Agency. Available at: <https://inis.iaea.org/records/4bq13-1cg98> (Accessed: 1 April 2026).

Tao, Q., Yu, J., Mu, X., Jia, X., Shi, R., Yao, Z., Wang, C., Zhang, H. and Liu, X. (2025) 'Machine learning strategies for small sample size in materials science', *Science China Materials*, 68(2), pp. 387–405. Available at: <http://dx.doi.org/10.1007/s40843-024-3204-5> (Accessed: 1 April 2026).

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J. and Van Der Walt, S. J. (2020) 'SciPy 1.0: fundamental algorithms for scientific computing in Python', *Nature Methods*, 17(3), pp. 261–272. Available at: <https://doi.org/10.1038/s41592-019-0686-2> (Accessed: 1 April 2026).

Zhang, R., Zhu, S., Wang, K., She, D., Argaud, J. P., Bouriquet, B., Li, Q. and Gong, H. L. (2025) 'Artificial Intelligence in Reactor Physics: Current Status and Future Prospects', (Compendex), pp. Available at: <http://dx.doi.org/10.48550/arXiv.2503.02440> (Accessed: 1 April 2026).

Zohuri, B., Balgehshiri, S. K. M. and Lomonaco, G. (2025) *Nuclear Reactor Physics and Operation: Neutronic Analysis for Nuclear Reactor Design*. CRC Press, p. Reprint. Available at: <http://dx.doi.org/10.1201/9781003590194> (Accessed: 1 April 2026).

Appendices

Appendix A: Project Package Layout and User Guides

A.1 Package Location

The complete three-stage pipeline used in this study has been archived for reproducibility within the following GitHub repository:

- **Repository Link:** <https://github.com/JADehghani/Project-Package-OpenMC-RFR-and-XGBoost-Models-of-a-NuScale-Like-SMR->
- **Note:** *More comprehensive documentation is detailed in the repository's `README.md` file.*

A.2 Package Components

The pipeline is divided into three sequential folders, directly following the methodology detailed in Ch. 3 of this report:

- **/1. OpenMC (Data Generation):** Contains the core builder script `build-core-fresh.py` and the overarching dataset generation script `generate_dataset.py`.
- **/2. Data Cleaning (Preprocessing):** Contains the Jupyter notebook `Raw_data_cleaner.ipynb` used to process raw OpenMC results for training tree-based ML algorithms.
- **/3. Surrogate Models (ML Training and Testing):** Houses the saved XGBoost and RFR models (`.pkl` files), alongside the test script `Surrogate_model_test.py`.
- **Note:** *All visual plots are saved to designated `/ * Plots` folders within their respective phases in the pipeline.*

A.3 Execution and User Guides

Each of the three primary folders contains a dedicated `User Guide.txt` file. These guides provide step by step instructions for executing the pipeline. Furthermore, each directory contains an `Outputs.txt` file detailing the terminal logs for all scripts, allowing external users assistance with the expected outputs in reproducing this study.

Appendix B: Updated Risk Register

Table 7: Final Project Risk Register.

Category	Identified Risk	Potential Impact	Mitigation Strategy
Baseline Data Generation	Computational Bottleneck	Monte Carlo simulations require excessive computation time to generate reliable training data.	Constrained the training dataset to 1 000 samples. Accepting the 16.8-hour compute time as a trade-off for achieving a high-fidelity (sub-100 <i>pcm</i>) training dataset.
	Statistical Noise	ML models learn incorrect correlations due to excessive error in the baseline data.	Ensured the base dataset achieved a sub-100 <i>pcm</i> uncertainty to reduce noise and ensured the best gridsearch hyperparameters successfully converge to a minimum.
	Hardware Failure Mid-Execution	PC crashes mid-execution, losing 16+ hours of simulation data.	Included exceptions in the data generation pipeline to ensure the results are saved if the process is interrupted mid-run.
Feature Selection/ Data Cleaning	Feature Definition Gaps	Selected input parameters don't correlate strongly enough with k_{eff} .	Grounded feature definition strictly to physical material properties that directly impact the four-factor formula.
	Base Data Truth	Base data doesn't accurately reflect true SMR physics.	Utilised a verified reactor model benchmark to ensure model accuracy and verified the base dataset against physics via a Spearman correlation heatmap.
Machine Learning	Model Overfitting/ Underfitting	Models successfully learn training data but fall short on unseen data.	Used 5-fold Cross-Validation paired with gridsearch to optimise hyperparameter tuning, ensuring the ML models are accurate on unseen data.
	The Black Box Dilemma	Models predict targets accurately, but underlying logic cannot be explained, violating ALARP safety principles.	Extracted feature hierarchies using the Gini impurity index and identified the black box grey area with tree-based models.
	Generative AI code hallucination	Generative AI produces inaccurate plotting scripts that misrepresent the results of the study.	Manually audited all AI-generated graph plotting scripts, verifying the outputs against summary statistics of the final comparison dataset.
Project Management	Scope Issues	Constant tweaking of project code delays final report writing.	Implemented boundaries to avoid chasing higher simulation fidelity in both the MC and ML models, which comes with diminishing returns as computation increases.
	Literature Gap	Gap in literature review could result in an ungrounded methodology.	Implemented a two-week literature review catchup after the interim assessment to ensure all research gaps were filled (following supervisor advice).

Appendix C: Initial vs Actual Project Gantt Charts

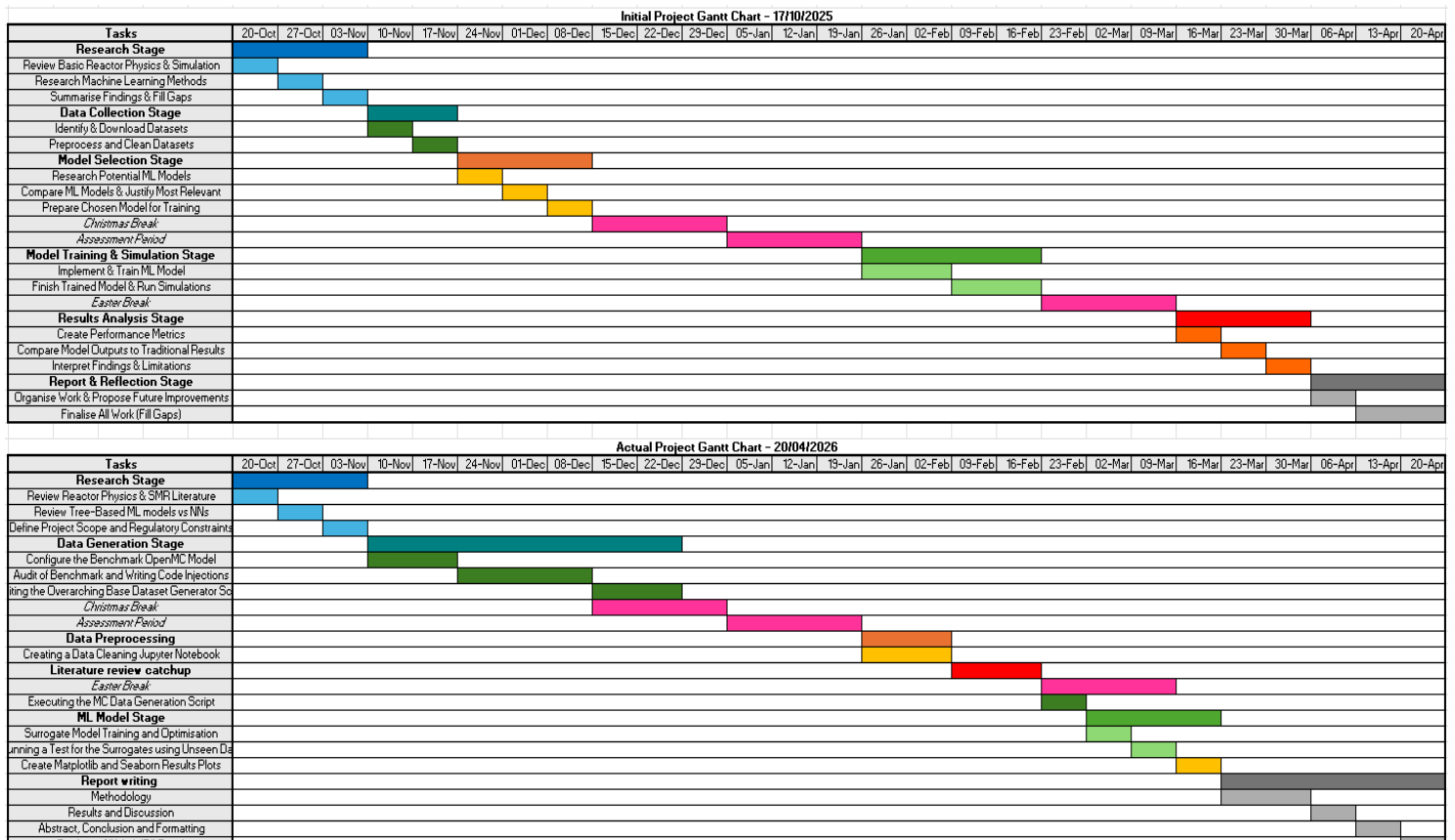


Figure 9: Initial and Actual Project Gantt Charts from the Project Proposal Stage (17/10/25) to Submission (24/04/26).

Appendix D: Generative AI Assisted Plots

D.1 Acknowledgments

In accordance with the University of Liverpool Academic Integrity Policy, this appendix acknowledges the permitted use of generative AI (Anthropic, 2026) strictly as an assistive tool. Claude.ai was utilised to generate high quality Python plotting scripts (using `Matplotlib` and `seaborn` libraries) for the graphical visualisations used in this report. All Python code related to the core three-stage pipeline remains as the work of the author.

Generative AI assisted in writing code for plot generation in the following scripts:

- `Raw_data_cleaner.ipynb`:
 - Dual K_{eff} distribution and MC uncertainty plots (Fig. 4)
 - Spearman correlation heatmap (Fig. 3)
- `XGB_surrogate.py` and `XGB_surrogate.py`:

- Feature importance bar charts (Fig. 5)
- **Surrogate_model_test.py:**
 - Dual parity plots (Fig. 6)
 - Dual error distribution violin plots (Fig. 7)
 - Dual breakdown of dominant features (Fig. 8)

D.2 Code Example

The following script is an example of one generated with the assistance of Claude.ai. This one specifically generates and saves the Gini impurity-based feature importance bar charts used in Sec. 4.3. The following prompt was used to generate the base code before it was tweaked and verified: “Generate a dissertation-quality Python plotting script using Matplotlib to visualise a bar chart for the feature importance of a RFR. Feature importance can be extracted using ‘best_rfr.feature_importance’. All labels and titles should be bold. The colour of the bars should be blue (#005A9C). Include comments for interpretability”. The code example below demonstrates how generative AI assisted in standardising the graph format:

```
# ---- Feature Importance Visualization ---- #
# Generates a bar chart displaying the relative importance of each reactor parameter in
# predicting k_eff. Gini impurity measures importance across all trees, providing insight
# into which features influence reactor criticality the most. Created with assistance
# from 'Claude.ai'.
# ----- #

importances = best_rfr.feature_importances_ # Extract Gini importance scores
indices = np.argsort(importances) # Sort features by importance (ascending)

# Configure global matplotlib font settings for consistency
plt.rcParams.update({
    'font.weight': 'bold',
    'axes.labelweight': 'bold',
    'axes.titleweight': 'bold',
    'figure.titleweight': 'bold'
})

plt.figure(figsize=(9, 6), facecolor='white')

# Create horizontal bar chart with navy blue color scheme
plt.barh(range(len(indices)), importances[indices], color='#005A9C', align='center',
         edgecolor='black', linewidth=0.5)

# Map feature names to clean presentation labels
clean_labels = {
    'Fuel_Temperature_K': 'Fuel Temp',
    'Cladding_Temperature_K': 'Clad Temp',
    'Moderator_Temperature_K': 'Mod Temp',
    'Boron_ppm': 'Boron Conc',
    'CR_Step': 'CR Step'
}

# Apply clean labels to y-axis
display_names = [clean_labels.get(feature_names[i], feature_names[i]) for i in indices]
plt.yticks(range(len(indices)), display_names, fontsize=11, fontname='sans-serif')
plt.xlabel('Relative Importance (Gini)', fontsize=12, fontname='sans-serif')
plt.title('A: RFR Feature Importance', fontsize=14, pad=15, fontname='sans-serif')

# Add subtle grid lines for improved readability
plt.grid(axis='x', linestyle='--', alpha=0.6, color='#A6A6A6')

# Remove top and right spines for clean style
plt.gca().spines['top'].set_visible(False)
plt.gca().spines['right'].set_visible(False)

# Output the plot and save it as a .png file
plt.tight_layout()
plt.savefig('Feature_Importance_RFR.png', dpi=300, bbox_inches='tight')
print("Saved Feature Importance Chart")
plt.show()
```

Listing 1: Code Snippet from RFR_surrogate.py for Generating the RFR Gini Importance plot using Matplotlib.